MODULE 01 · PYTHON + ASYNC + FASTAPI FOR LLM ENGINEERING · DAYS 01–05

Python Foundations for AI Engineering: Days 01–05

One guide to the first five notebooks of Module 01. Each Day explains the idea in plain words, shows a workflow picture where it helps, and walks through the notebook's own examples with their real output.

PA

Tutor: Prudhvi Akella

Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Notebooks: 01_Variables_Types · 02_Conditions_Loops · 03_Functions_DataStructures · 04_File_IO_JSON · 05_Exception_Handling_Logging_OpenTelemetry

Running example: **ShopSmart**, an online store whose AI support assistant reads customer data, routes questions, builds prompts, calls an LLM and logs the result. Python 3.10+

Start here

1 How the five days fit together
2 How to read the examples

Day 01 · Variables, Types & f-strings

3 Set up a project: the virtual environment
4 Variables: a name for a value
5 The five basic types
6 Type conversion
7 f-strings: building prompts from data
8 Four containers for many values
9 list: an ordered collection
10 dict: look things up by name
11 Nested data: drilling into API replies
12 set: unique values and fast membership checks
13 tuple: values that belong together and don't change
14 Strings are sequences too
15 Day 01 recap

Day 02 · Conditions & Loops

16 Conditions: questions with a True / False answer
17 if / elif / else: choose one path
18 Use case: route a customer query to the right agent
19 for loops: do something for every item
20 enumerate: the position and the item together
21 Loop + if: turn database values into customer messages
22 range: counting without a list
23 Use case: process queries in batches
24 Accumulator: collect results from a loop
25 while loops: repeat until a condition changes
26 break and continue
27 Use case: pick few-shot examples for a prompt
28 List comprehensions: build a list in one line
29 Dict comprehensions
30 Day 02 recap

Day 03 · Functions & Data Structures

31 Functions: write once, reuse anywhere
32 Use case: calculate a discount
33 Default parameter values
34 *args: any number of positional values
35 **kwargs: any number of named settings
36 Lists and their everyday methods
37 Use case: build chat message history
38 Dictionaries: reading values safely
39 Dictionary methods
40 Use case: index products by name
41 Sets: unique values
42 Tuples and zip()
43 Day 03 recap and practice

Day 04 · File I/O, JSON & CSV

44 What JSON is
45 json.loads() and json.dumps(): strings
46 json.dump() and json.load(): files
47 with open(): safe file handling
48 Use case: load a system prompt from a file
49 Use case: log every LLM call (JSONL)
50 Use case: parse JSON returned by an LLM
51 Read CSV rows with csv.DictReader
52 Other CSV shapes
53 Putting it together: a small AI workflow
54 Day 04 recap

Day 05 · Exceptions, Logging & Retry

55 What is an exception?
56 try and except
57 Catch the specific errors you expect
58 Read the error message with `as`
59 finally: code that always runs
60 raise: report invalid input yourself
61 Logging instead of print
62 Use case: one bad CSV row must not stop the job
63 Retry with exponential backoff
64 A first look at OpenTelemetry (optional)
65 Day 05 recap and practice

Wrap-up · All five days in one script

66 The script
67 Read it like this

1 How the five days fit together

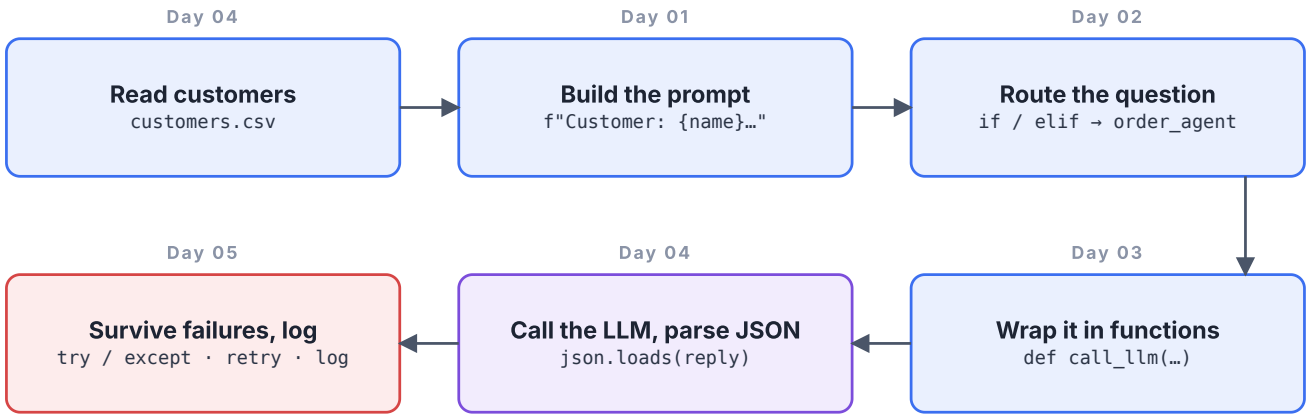


Figure 0.1 One customer question, end to end. Each step uses what one Day teaches.

Each notebook adds one skill. By Day 05 you can write the small program behind an AI support assistant: read a customer from a file, build a prompt, pick the right agent, call the model, read its JSON reply, and keep going when something fails. The last section of this guide puts all five days into one script.

Day	Notebook	In one line
01	Variables, Types & f-strings	Store values, build prompts, hold many values in list / dict / set / tuple
02	Conditions & Loops	Make decisions and repeat work over many items
03	Functions & Data Structures	Package code to reuse it; the everyday list and dict methods
04	File I/O, JSON & CSV	Read and write files; turn JSON and CSV into Python data
05	Exceptions, Logging & Retry	Handle errors, record what happened, retry temporary failures

2 How to read the examples

Examples are copied from the notebooks. The small grey label shows the notebook cell number, so you can find it. Long comment blocks such as `# ITERATION TRACE` are shown as tables or pictures instead.

Blue block = the code you type. Green block = what Python printed when the notebook ran. Coloured notes: **blue** = key idea · **yellow** = watch out · **green** = tip · **red** = common mistake. Learn by typing: run each example, change one value, predict the output, run again.

DAY 01

Variables, Types & f-strings

How Python stores data: single values (numbers, text, True/False, None), how to build LLM prompts from them with f-strings, and the four containers — list, dict, set, tuple — that hold many values.

Notebook: **01_Variables_Types.ipynb**

Project setup | Variables | 5 basic types | Type conversion | f-strings | list | dict | Nested data | set | tuple | String indexing

3 Set up a project: the virtual environment



Figure 1.1 Each project gets its own private box of packages, so projects never break each other.

Before writing any code, each project gets a **virtual environment** (`venv`). Think of it as a private toolbox for one project. Packages you install go into that box only, so Project A can use a new library while Project B keeps an old one. Run these commands **in the terminal**, not inside Python:

Step	Mac / Linux	Windows (PowerShell)
1. Create the box	`python3 -m venv venv`	`python -m venv venv`
2. Step inside it	`source venv/bin/activate`	`venv\Scripts\activate`
3. Install packages	`pip install -r requirements.txt`	`pip install -r requirements.txt`
4. Run your code	`python modules/day01_setup_variables.py`	`python modules\day01_setup_variables.py`

How do you know it worked? Your terminal line starts with `(venv)` . Every `pip install` now lands in the project's `venv` folder, not in the Python installed on your laptop.

4 Variables: a name for a value

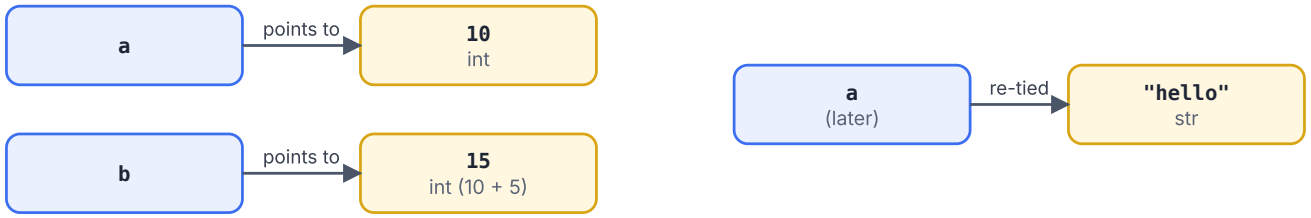


Figure 1.2 A variable is a name tag tied to a value. Python works out the right side first, then ties the name to the result.

A **variable** is a name that points to a value, so you can use the value again later. In `b = a + 5` Python reads the **right side first** (`a + 5` → `15`) and only then stores the result under the name `b`.

NOTEBOOK CELL 18

```
a = 10          # right side: 10. Stored as int.
b = a + 5       # right side: 10+5=15. Stored in b.
c = b * 2       # right side: 15*2=30. Stored in c.

# You can reassign any name to a different type
a = "hello"     # a now points to a str — Python allows this
print(a, type(a))
```

OUTPUT

```
hello <class 'str'>
```

> Python does not lock a name to a type. `a` held a number, then text. This is allowed, but in real projects keep one meaning per name — it makes code easier to read.

5 The five basic types

Every value has a **type**. The type decides what you can do with it — you can add two numbers, but you cannot add a number to text. The notebook uses one LLM example for each type:

Type	Holds	Notebook example	Check it
`int`	Whole numbers	`max_tokens = 1024` , `customer_id = 1001`	`type(1024)` → `int`
`float`	Numbers with a decimal point	`temperature = 0.2` , `price = 205.21`	`type(0.2)` → `float`
`str`	Text, in quotes	`model_name = "gpt-4o"` , `order_id = "ORD-3042"`	`len("Danielle Johnson")` → `16`
`bool`	`True` or `False` only	`stream_response = True`	`10 > 5` → `True`
`None`	"No value yet"	`api_key = None` (loaded later)	`x is None` → `True`

Numbers: int and float

NOTEBOOK CELL 6

```
import json

# int: no decimal point, can be positive or negative
max_tokens  : int = 1024   # max LLM response length
max_retries : int = 3      # retry on failure
customer_id : int = 1001   # from customers.csv

print(type(max_tokens))    # <class 'int'>
print(10 + 3)              # 13
print(10 // 3)             # 3  — floor division (drops remainder)
print(10 % 3)              # 1  — modulo (remainder only)
```

OUTPUT

```
<class 'int'>
13
3
1
```

Operator	Meaning	Example
`/`	Divide — **always** gives a float	`10 / 3` → `3.3333333333333335`
`//`	Floor division — drop the remainder	`10 // 3` → `3`
`%`	Modulo — keep only the remainder	`10 % 3` → `1`

Text, True/False and None

NOTEBOOK CELL 10

```
# str: any characters in single or double quotes — both identical
model_name    : str = "gpt-4o"
customer_name : str = "Danielle Johnson"
order_id      : str = "ORD-3042"

print(type(model_name))    # <class 'str'>
print(len(customer_name))  # 16 — number of characters
```

OUTPUT

```
<class 'str'>
16
```

NOTEBOOK CELL 14

```
# None: represents 'not set yet' — check with  is None
api_key          = None   # loaded from .env later
pending_response = None   # LLM hasn't replied yet

print(pending_response is None)  # True  — correct way to check
print(type(None))                # <class 'NoneType'>
```

OUTPUT

```
True
<class 'NoneType'>
```

The `: int` in `max_tokens : int = 1024` is a **type hint** — a label for humans and editors. Python does not stop you from storing other types in it.

`True`, `False` and `None` start with a capital letter. `true` or `none` give an error. Check for None with `is None`, not `== None`.

6 Type conversion

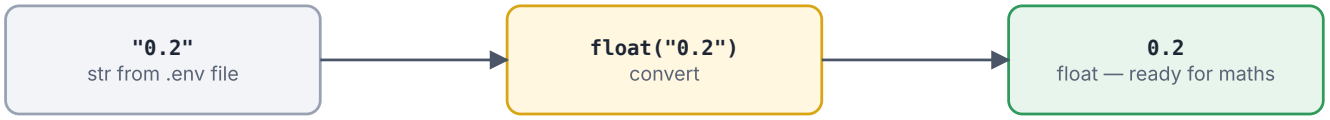


Figure 1.3 Data from files, forms and environment settings arrives as text. Convert it before doing maths.

Use the type's name as a function to convert a value: `str()`, `int()`, `float()`, `bool()`.

NOTEBOOK CELL 21

```
# str()
print(str(1024))       # '1024'
print(str(True))       # 'True'

# int() — truncates, does NOT round
print(int('1024'))     # 1024
print(int(3.9))        # 3  ← truncated, not 4

# float() — essential for env vars
print(float('0.2'))    # 0.2
print(float(10))       # 10.0

# bool() — falsy values
print(bool(0))         # False
print(bool(''))        # False — empty string
print(bool([]))        # False — empty list
print(bool('hello'))   # True  — non-empty
```

OUTPUT

```
1024
True
1024
3
0.2
10.0
False
False
False
True
```

Function	Watch out for
`int(3.9)`	Gives `3`. It **cuts off** the decimal part; it does not round.
`int("3.9")`	Error — text must look like a whole number. Use `float("3.9")` first.

Function	Watch out for
`bool(x)`	Empty things are `False`: `0`, `""`, `[]`, `{}`, `None`. Everything else is `True`.

You will use this every day: environment variables (Day 01 setup) and CSV files (Day 04) always give you **text**. `"1001" > "200"` is `False` because text compares letter by letter. Convert first.

7 f-strings: building prompts from data

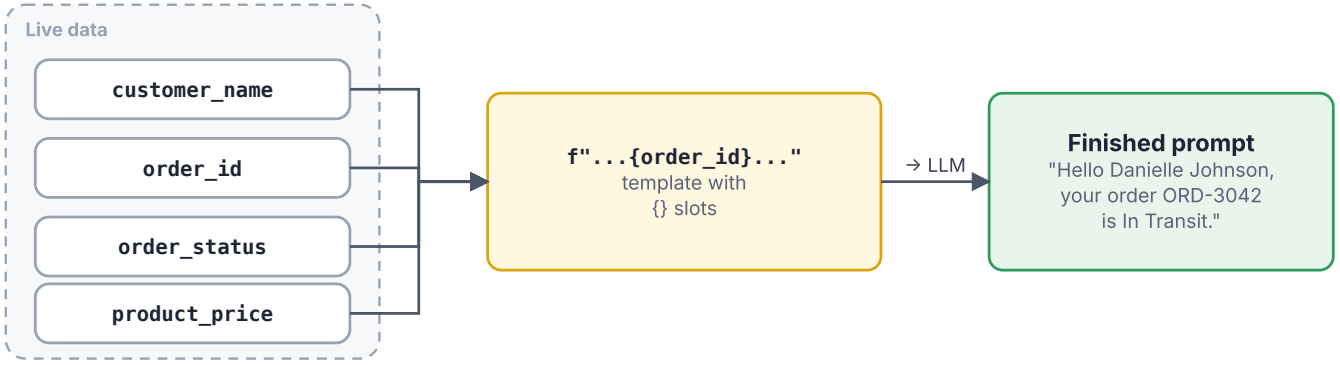


Figure 1.4 An f-string is a prompt template. Values from variables drop into the {} slots at run time.

The notebook calls this **the most important concept of Day 01**. An LLM prompt is just text, but the customer name, order and status change for every request. Put `f` before the quotes and write variable names inside `{}` — Python fills them in.

NOTEBOOK CELL 25

```
customer_name = "Danielle Johnson"
order_id      = "ORD-3042"
order_status  = "In Transit"
product_price = 205.21

# Basic f-string
msg = f"Hello {customer_name}, your order {order_id} is {order_status}."
print(msg)

# Number formatting
print(f"{1048576:,}")        # 1,048,576  — thousand separator
print(f"{product_price:.2f}") # 205.21     — 2 decimal places
print(f"{0.19999:.1f}")       # 0.2        — 1 decimal place
```

OUTPUT

```
Hello Danielle Johnson, your order ORD-3042 is In Transit.
1,048,576
205.21
0.2
```

Formatting inside the braces

Write	Means	Result
`{1048576:,}`	Add thousand separators	`1,048,576`
`{205.21:.2f}`	Exactly 2 decimal places (rounds)	`205.21`

Write	Means	Result
`{0.19999:.1f}`	1 decimal place	`0.2`
`{status:12s}`	Text padded to 12 characters (used on Day 02)	`Delivered···`
`{salary:>10.1f}`	Right-aligned in 10 characters, 1 decimal	`···25000.0`

Multi-line prompts

Triple quotes `f""" ... """` let one prompt span many lines — exactly how you write a real LLM prompt. Here `product_price` was `205.2167888`; `:.2f` rounds it to `205.22`.

NOTEBOOK CELL 27

```
# Multi-line f-string — the LLM prompt pattern
user_message = f"""\
Customer : {customer_name}
Order    : {order_id}
Status   : {order_status}
Amount   : ${product_price:.2f}

Question: Where is my order and when will it arrive?
Answer in 2 sentences maximum.
"""
print(user_message)
```

OUTPUT

```
Customer : Prudhvi Akella
Order    : ORD-3045
Status   : Delivered
Amount   : $205.22

Question: Where is my order and when will it arrive?
Answer in 2 sentences maximum.
```

Spaces at the start of each line are **kept**. The notebook's next cell (cell 29) indents the template, so every prompt line starts with 4 spaces that are sent to the LLM. Keep the lines of a multi-line f-string at the left edge, as in cell 27.

8 Four containers for many values



Figure 1.5 Four containers, four shapes. Pick by the question you will ask of the data.

Container	Syntax	Key property	LLM use in the notebook
`list`	`[...]`	Ordered, can change, can mix types	Conversation history (`messages`)

Container	Syntax	Key property	LLM use in the notebook
`dict`	`{key: value}`	Look up a value by its key	Model settings, one database row, an API reply
`set`	`{...}`	No duplicates, very fast "is it in?" check	Which tools an AI agent may use
`tuple`	`(...)`	Ordered, **cannot change**	Fixed settings, a function returning 2+ values

9 list: an ordered collection

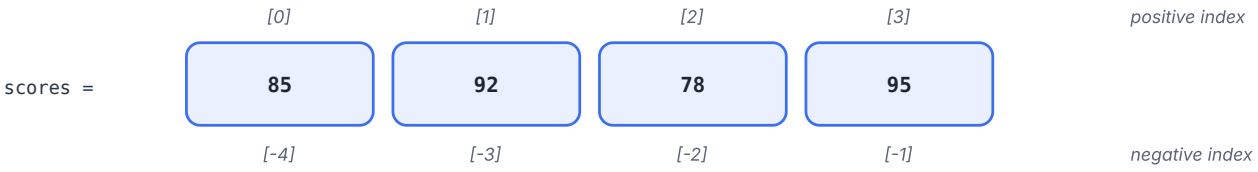


Figure 1.6 Positions start at 0. Negative positions count from the end.

A **list** keeps items in order. Get one item by its **position** (index) and a range of items with a **slice** `[start:end]` — the end position is not included, so `scores[1:3]` gives positions 1 and 2.

NOTEBOOK CELL 32

```
# Technically heterogeneous — Python allows mixed types
mixed = [1, "hello", 3.14, True]   # valid, but avoid this

# By convention: keep lists homogeneous
scores  = [85, 92, 78, 95]         # all int
models  = ["gpt-4o", "gpt-3.5"]   # all str

print(scores[0])    # 85  — first item
print(scores[-1])   # 95  — last item
print(scores[1:3])  # [92, 78] — slice
```

OUTPUT

```
85
95
[92, 78]
```

Lists can be changed after they are created (they are **mutable**):

NOTEBOOK CELL 33

```
print(scores)
scores[2] = 75
print(scores)
```

OUTPUT

```
[85, 92, 78, 95]
[85, 92, 75, 95]
```

Python allows mixed lists like `[1, "hello", 3.14, True]`, but by convention keep one kind of item per list. A list of messages, a list of scores — code that loops over it can then treat every item the same way.

10 dict: look things up by name

A **dictionary** stores **key → value** pairs. Instead of asking for "position 2" you ask for `"temperature"`. Model settings are a natural fit:

NOTEBOOK CELL 35

```
model_config = {
    "model"       : "gpt-4o",
    "max_tokens"  : 1024,
    "temperature": 0.2,
    "stream"      : False,
}

print(model_config["model"])       # gpt-4o
print(model_config["temperature"]) # 0.2
```

OUTPUT

```
gpt-4o
0.2
```

A database row is a dict; a table is a list of dicts

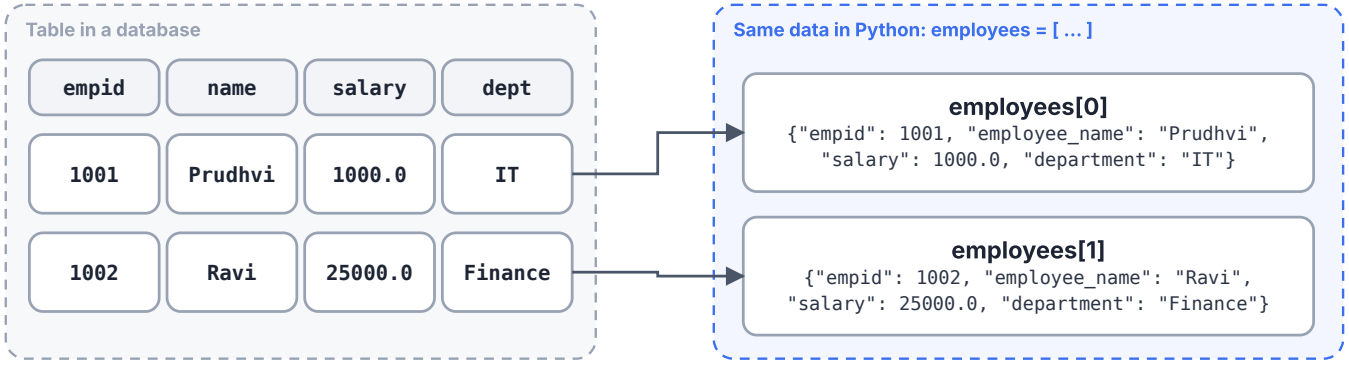


Figure 1.7 A database table in Python: each row is a dict (column → value) and the table is a list of those dicts.

NOTEBOOK CELL 38

```
# Each row becomes one dict — column name = key, cell value = value
row1 = {
    "empid"         : 1001,       # int
    "employee_name" : "Prudhvi",  # str
    "salary"        : 1000.0,     # float
    "address"       : "RGT",      # str
    "department"    : "IT",       # str
}

row2 = {
    "empid"         : 1002,
    "employee_name" : "Ravi",
    "salary"        : 25000.0,
    "address"       : "HYD",
    "department"    : "Finance",
}

# Full table = list of dicts
employees = [row1, row2]

print(employees[0]["employee_name"])  # Prudhvi
print(employees[1]["salary"])          # 25000.0
print(len(employees))                  # 2 — number of rows
```

OUTPUT

```
Prudhvi
25000.0
2
```

Loop over the rows to print a report, or keep only some rows — the same thing a SQL `WHERE` does:

NOTEBOOK CELL 41

```
# Loop over all rows — the standard DB query pattern
for emp in employees:
    print(f"{emp['empid']}  {emp['employee_name']:15s}  {emp['salary']:>10.1f}
{emp['department']}")

# Filter: only IT department
it_team = [emp for emp in employees if emp["department"] == "IT"]
print(it_team[0]["employee_name"])     # Prudhvi
```

OUTPUT

```
1001  Prudhvi             1000.0  IT
1002  Ravi               25000.0  Finance
Prudhvi
```

Dict values drop straight into an f-string, so one row becomes a prompt or a report (cell 40):

NOTEBOOK CELL 40

```
template = f"""
Employee Details:
Name : {employees[1]["employee_name"]}
Salary : {employees[1]["salary"]}
Address : {employees[1]["address"]}
Department : {employees[1]["department"]}
"""

print(template)
```

OUTPUT

```
Employee Details:
Name : Ravi
Salary : 25000.0
Address : HYD
Department : Finance
```

11 Nested data: drilling into API replies

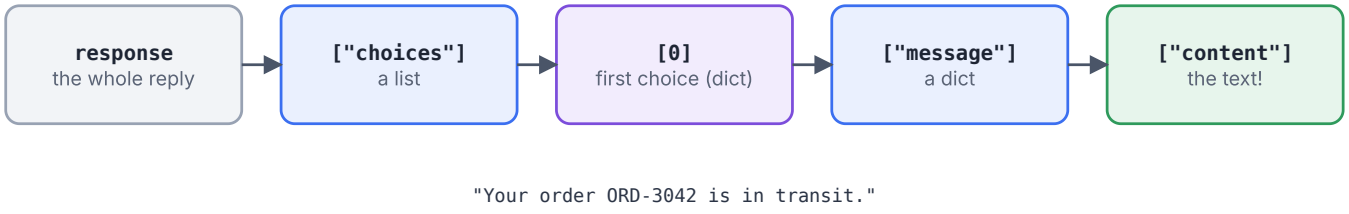


Figure 1.8 Reading a nested value is a chain of steps: key → position → key → key.

Real data has dicts inside lists inside dicts. A customer has an address (a dict) and many orders (a list of dicts). Read it one step at a time, left to right:

NOTEBOOK CELL 43

```
# A customer record — nested dict (this is what a DB row looks like in Python)
customer = {
    "id"   : 1001,
    "name" : "Danielle Johnson",
    "address": {
        "city"   : "Hyderabad",
        "state"  : "Telangana",
        "pincode": "500001"
    },
    "orders": [
        {"order_id": "ORD-3042", "status": "In Transit", "amount": 205.21},
        {"order_id": "ORD-2891", "status": "Delivered",  "amount": 568.17}
    ]
}

# Drill into nested values
print(customer["name"])                    # Danielle Johnson
print(customer["address"]["city"])          # Hyderabad
print(customer["orders"][0]["order_id"])    # ORD-3042
print(customer["orders"][1]["status"])      # Delivered
```

OUTPUT

```
Danielle Johnson
Hyderabad
ORD-3042
Delivered
```

The same pattern on an LLM API reply

An OpenAI-style response is a nested dict. Two lines get the reply text and the token count:

NOTEBOOK CELL 44

```
# The OpenAI API response is also a nested dict — same pattern
response = {
    "model": "gpt-4o",
    "choices": [
        {
            "message": {
                "role"   : "assistant",
                "content": "Your order ORD-3042 is in transit."
            }
        }
    ],
    "usage": {
        "prompt_tokens"    : 45,
        "completion_tokens": 12,
        "total_tokens"     : 57
    }
}
response["choices"][0]['message']['content'] # "Your order ORD-3042 is in transit."
# Get the LLM reply text:
reply  = response["choices"][0]["message"]["content"]
tokens = response["usage"]["total_tokens"]

print(reply)   # Your order ORD-3042 is in transit.
print(f"Used {tokens} tokens")
```

OUTPUT

```
Your order ORD-3042 is in transit.
Used 57 tokens
```

Cell 39 does the same with employees that have several addresses. Breaking the chain into steps helps when you are unsure:

```
employee_3_address = employee_3["address"]          # the list of addresses
first_address      = employee_3_address[0]         # the first address (a dict)
print(first_address["pincode"])                    # 59108

print(employee_3["address"][0]["pincode"])         # same thing in one line → 59108
```

If you get a `KeyError` or `IndexError`, print each step of the chain separately. One of the keys is spelt differently or the list is shorter than you expect.

12 set: unique values and fast membership checks

A **set** keeps each value **once** and has no order (so no `[0]`). Its superpower is the question "is X in here?" — answered instantly, whether the set has 5 or 50,000 items.

NOTEBOOK CELL 48

```
# ── Creating a set ──────────────────────────────────────────

# 1. Literal syntax — curly braces with values
fruits = {"apple", "banana", "cherry"}
print(fruits)           # unordered — output order may vary
print(type(fruits))     # <class 'set'>

# 2. Duplicates are automatically removed
tags = {"IT", "Finance", "IT", "HR", "Finance", "IT"}
print(tags)             # {'IT', 'Finance', 'HR'} — only 3 items

# 3. Convert a list to a set (remove duplicates)
departments_list = ["IT", "Finance", "IT", "HR", "Finance"]
departments_set  = set(departments_list)
print(departments_set)  # {'IT', 'Finance', 'HR'}

# 4. Empty set — use set(), NOT {} which creates an empty dict
empty_set  = set()      # correct
empty_dict = {}         # this is a dict, not a set!
print(type(empty_set))  # <class 'set'>
print(type(empty_dict)) # <class 'dict'>
```

OUTPUT

```
{'banana', 'apple', 'cherry'}
<class 'set'>
{'IT', 'Finance', 'HR'}
{'IT', 'Finance', 'HR'}
<class 'set'>
<class 'dict'>
```

`{}` creates an empty **dict**, not a set. For an empty set write `set()`. Set output order can change from run to run — never rely on it.

Use case: an AI agent's safety gate

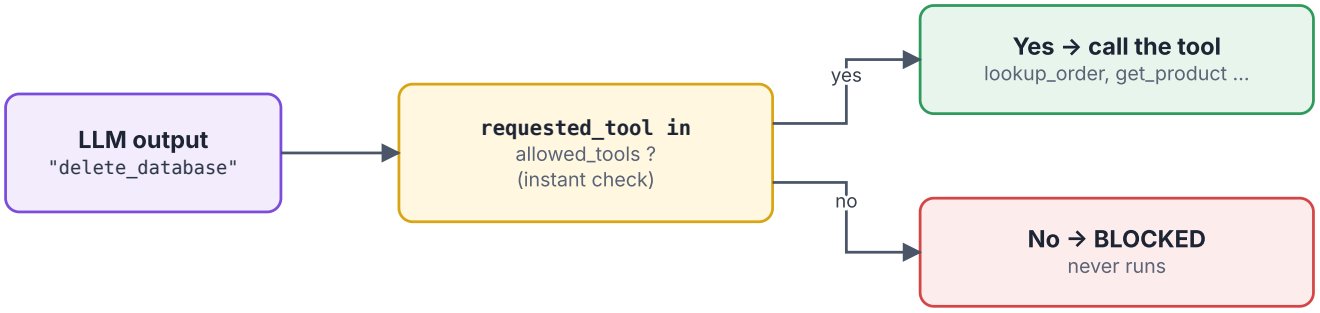


Figure 1.9 An agent safety gate: the LLM suggests a tool, a set check decides if it may run.

NOTEBOOK CELL 50

```
# ── Membership check — O(1) — the PRIMARY use of sets ────────

allowed_tools = {"lookup_order", "lookup_customer", "check_inventory", "get_product"}

print("lookup_order"    in allowed_tools)   # True
print("delete_database" in allowed_tools)   # False
print("send_email"      in allowed_tools)   # False

# ── LLM Agent safety gate ──────────────────────────────────
# The LLM decides which tool to call — we must validate it
requested_tool = "delete_database"   # LLM output

if requested_tool in allowed_tools:
    print(f"Calling tool: {requested_tool}")
else:
    print(f"BLOCKED — '{requested_tool}' is not an allowed tool")

# This check is O(1) whether allowed_tools has 5 or 50,000 entries
```

OUTPUT

```
True
False
False
BLOCKED — 'delete_database' is not an allowed tool
```

Add and remove

NOTEBOOK CELL 51

```
# ── Adding and removing items ──────────────────────────────

tools = {"lookup_order", "lookup_customer"}

tools.add("check_inventory")     # add one item
print(tools)

tools.remove("lookup_customer")  # remove — raises KeyError if not found
tools.discard("send_email")      # remove safely — no error if missing
print(tools)

print(len(tools))   # number of items in the set
```

OUTPUT

```
{'lookup_order', 'check_inventory', 'lookup_customer'}
{'lookup_order', 'check_inventory'}
2
```

Method	If the item is missing
`.add(x)`	Adds it (if already there, nothing happens)
`.remove(x)`	Raises `KeyError`
`.discard(x)`	Does nothing — the safe way to remove

Compare two groups

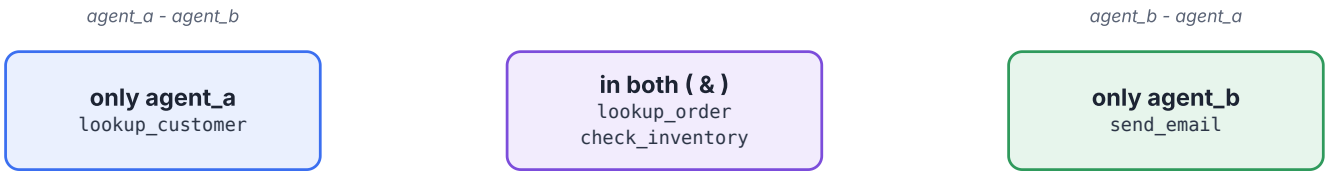


Figure 1.10 Set operations on the tools of two agents.

NOTEBOOK CELL 52

```
# ── Set operations — mathematics in Python ────────────────────

agent_a = {"lookup_order", "lookup_customer", "check_inventory"}
agent_b = {"lookup_order", "send_email",     "check_inventory"}

combined_output = agent_a | agent_b
print("Union  (|) — all tools from both:",     combined_output)
intersection_output = agent_a & agent_b
print("Intersect (&) — tools in both agents:",  intersection_output)
difference_in_refernce_agenta_output = agent_a - agent_b
print("Difference (-) — only in agent_a:",      difference_in_refernce_agenta_output)
difference_in_refernce_agentb_output = agent_b - agent_a
print("Difference (-) — only in agent_b:",      difference_in_refernce_agentb_output)
symentric_diff = agent_a ^ agent_b
print("Sym diff (^) — in one but not both:",    symentric_diff)
```

OUTPUT

```
Union  (|) — all tools from both: {'lookup_order', 'check_inventory', 'send_email',
'lookup_customer'}
Intersect (&) — tools in both agents: {'lookup_order', 'check_inventory'}
Difference (-) — only in agent_a: {'lookup_customer'}
Difference (-) — only in agent_b: {'send_email'}
Sym diff (^) — in one but not both: {'lookup_customer', 'send_email'}
```

Rule of thumb from the notebook: asking **"is X in this group?"** → use a set. Asking **"what is the Nth item?"** or the order matters (chat history) → use a list.

13 tuple: values that belong together and don't change

A **tuple** is like a list that cannot be changed after it is made. Use it for values that belong together: a coordinate, a fixed config, a function's several results.

NOTEBOOK CELL 55

```
# ── Creating a tuple ──────────────────────────────────────

# 1. With parentheses
coords  = (17.3850, 78.4867)          # Hyderabad lat/lon
version = (4, "Added few-shot examples")
config  = ("gpt-4o", 1024, 0.2, True) # model, tokens, temp, stream

# 2. Without parentheses — Python still makes a tuple
point = 10, 20       # same as (10, 20)
print(type(point))   # <class 'tuple'>

# 3. Single-element tuple — the comma is required!
single   = (42,)     # tuple with one item
not_tuple = (42)     # just an int in parentheses
print(type(single))   # <class 'tuple'>
print(type(not_tuple))# <class 'int'>

# 4. Access by index — same as list
print(config[0])  # gpt-4o
print(config[1])  # 1024
print(config[-1]) # True
```

OUTPUT

```
<class 'tuple'>
<class 'tuple'>
<class 'int'>
gpt-4o
1024
True
```

A one-item tuple needs a comma: `(42,)`. Without it, `(42)` is just the number 42 in brackets.

To "change" a tuple you build a new one; the original stays as it was:

NOTEBOOK CELL 56

```
# ── Immutability — cannot change after creation ──────────

config = ("gpt-4o", 1024, 0.2)

config_v2 = ("gpt-3.5",) + config[1:]   # replace first item
print(config_v2)   # ('gpt-3.5', 1024, 0.2)
print(config)      # ('gpt-4o', 1024, 0.2) — original unchanged
```

OUTPUT

```
('gpt-3.5', 1024, 0.2)
('gpt-4o', 1024, 0.2)
```

Unpacking — the most used tuple feature

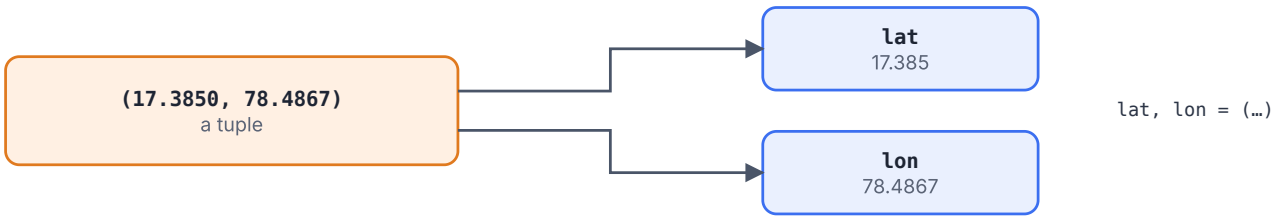


Figure 1.11 Unpacking: one tuple, several names, in one line.

NOTEBOOK CELL 57

```
# ── Unpacking — the most used tuple feature ──────────────────

# Basic unpacking — one variable per value
lat, lon = (17.3850, 78.4867)
print(f"Lat: {lat}, Lon: {lon}")

version_num, version_desc = (4, "Added few-shot examples")
print(version_num)    # 4
print(version_desc)   # Added few-shot examples

# Extended unpacking — capture the rest in a list
first, *rest = (1, 2, 3, 4, 5)
print(first)   # 1
print(rest)    # [2, 3, 4, 5]

# Swap two variables — classic Python trick (uses tuple internally)
a, b = 10, 20
a, b = b, a    # no temp variable needed
print(a, b)    # 20  10
```

OUTPUT

```
Lat: 17.385, Lon: 78.4867
4
Added few-shot examples
1
[2, 3, 4, 5]
20 10
```

Functions that return several values

When a function writes `return a, b, c`, Python packs them into a tuple. The caller unpacks them:

NOTEBOOK CELL 60

```
# ── Functions returning multiple values ──────────────────────
# Python returns multiple values as a tuple automatically

def get_model_config():
    return "gpt-4o", 1024, 0.2   # Python packs these into a tuple

result = get_model_config()
print(result)           # ('gpt-4o', 1024, 0.2)
print(type(result))     # <class 'tuple'>

# Unpack immediately
model, max_tokens, temp = get_model_config()
print(model)            # gpt-4o
print(max_tokens)       # 1024

# Another common pattern — min and max together
def salary_range(employees):
    salaries = [e["salary"] for e in employees]
    return min(salaries), max(salaries)   # returns tuple

employees = [{"name": "Prudhvi", "salary": 1000.0},
             {"name": "Ravi",    "salary": 25000.0}]
low, high = salary_range(employees)
print(f"Salary range: {low} – {high}")
```

OUTPUT

```
('gpt-4o', 1024, 0.2)
<class 'tuple'>
gpt-4o
1024
Salary range: 1000.0 – 25000.0
```

Tuples as dictionary keys: an LLM response cache

Dict keys must never change. Tuples can't change, so they can be keys — lists cannot. Here the pair (model, prompt) remembers earlier answers so you don't pay for the same API call twice:

NOTEBOOK CELL 61

```
cache = {
    ("gpt-4o",   "Where is order ORD-3042?") : "Your order is in transit.",
    ("gpt-4o",   "What is my balance?")       : "Your balance is $205.21.",
    ("gpt-3.5t", "Where is order ORD-3042?") : "Order ORD-3042 is in transit.",
}

key   = ("gpt-4o", "Where is order ORD-3042?")
reply = cache.get(key, "Not cached — call the API")
print(reply)   # Your order is in transit.

# Using a list as key raises TypeError:
# cache[["gpt-4o", "prompt"]]  ->  TypeError: unhashable type: 'list'
```

OUTPUT

```
Your order is in transit.
```

Choose	When	Example
`list`	Order matters and it grows or changes	`messages = [msg1, msg2, msg3]`

Choose	When	Example
`set`	Unique values, fast "is it in?"	`allowed = {"tool_a", "tool_b"}`
`tuple`	Fixed, values belong together	`config = ("gpt-4o", 1024, 0.2)`
`dict`	Look up by a name	`model_config["temperature"]`

14 Strings are sequences too

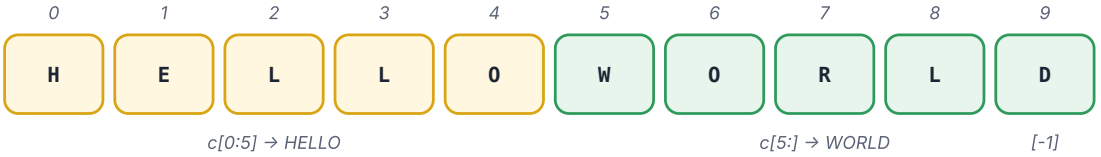


Figure 1.12 A string is a sequence of characters, indexed exactly like a list.

Everything you learned about list positions and slices works on text. Strings, like tuples, cannot be changed in place — build a new one instead:

NOTEBOOK CELL 66

```
c = "HELLOWORLD"

print(c[0])      # H   — first character
print(c[6])      # O   — positive index
print(c[-1])     # D   — last character
print(c[0:5])    # HELLO — slice
print(c[5:])     # WORLD — from index 5 to end
print(len(c))    # 10

new_c = "J" + c[1:]
print(new_c)     # JELLOWORLD
```

```
OUTPUT
H
O
D
HELLO
WORLD
10
JELLOWORLD
```

15 Day 01 recap

Concept	Remember
venv	One private package box per project. Look for `(venv)` in the terminal
Variable	A name tied to a value. The right side is worked out first
int / float / str / bool / None	Whole number / decimal / text / True-False / no value yet
Type conversion	`int()` cuts decimals, `bool()` of empty things is False; file and env data is text

Concept	Remember
f-string	`f"Hello {name}"` — the way to build LLM prompts. `:.2f`, `:,`, `:12s` format values
list	Ordered, changeable; `[0]`, `[-1]`, `[1:3]`
dict	key → value; a DB row. A table = list of dicts
Nested data	Chain the steps: `response["choices"][0]["message"]["content"]`
set	Unique values, instant `in` check; `| & - ^`
tuple	Fixed; unpack with `a, b = …`; can be a dict key

DAY 02

Conditions & Loops

Teach the program to decide (if / elif / else) and to repeat (for, while). The GenAI use cases: route a customer question to the right agent, process queries in batches, collect results, and pick few-shot examples.

Notebook: **02_Conditions_Loops.ipynb**

Conditions | Query routing | for | enumerate | range | Batching | Accumulator | while | break / continue | Few-shot selection | Comprehensions

16 Conditions: questions with a True / False answer

A **condition** is a question Python can answer with `True` or `False`: "is the price at most 1000?". Combine several with `and`, `or`, `not`.

Operator	Meaning	Example → result
`==` `!=`	equal / not equal	`3 == 7` → `False`
`>` `<` `>=` `<=`	bigger / smaller	`rating >= 4.0` → `True` when rating is 4.2
`in`	is it inside?	`"cancel" in "i want to cancel"` → `True`
`and`	both must be True	`stock > 0 and rating >= 4`
`or`	at least one True	`"cancel" in q or "refund" in q`
`not`	flip True ↔ False	`not is_recommended`

NOTEBOOK CELL 4

```
price = 800
stock = 10
rating = 4.2

# Relational Expression when its get evaluated the output is True/False
is_recommended = (price <= 1000 and stock > 0 and rating >= 4.0)
print(is_recommended)
```

OUTPUT

```
True
```

A condition on its own just gives True or False. Tied to `if`, it decides which code runs:

NOTEBOOK CELL 5

```
if is_recommended:
    print("Recommended Product")
else:
    print("Not recommended")
```

OUTPUT

```
Recommended Product
```

17 if / elif / else: choose one path

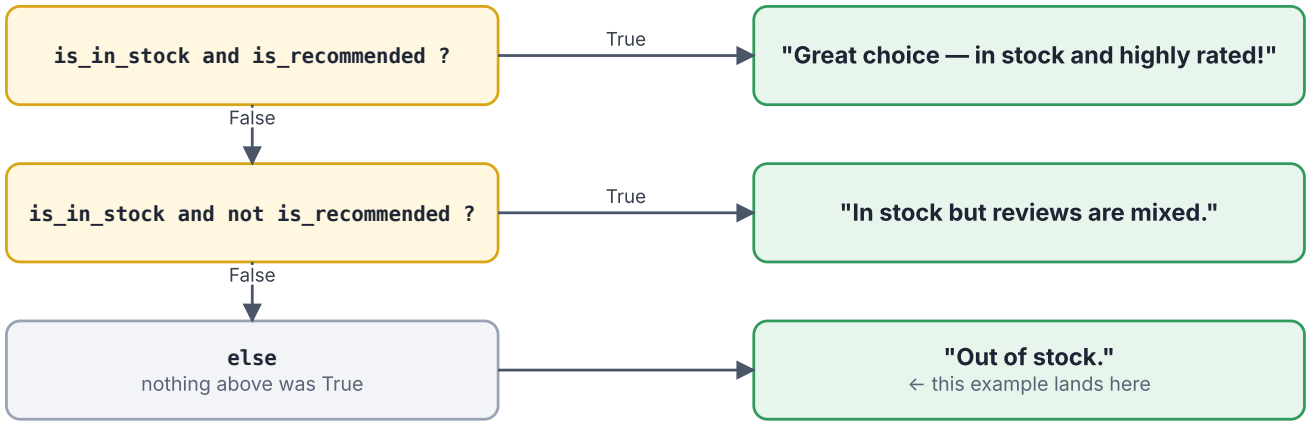


Figure 2.1 Python checks from the top. The first True branch runs; the rest are skipped.

Rule	Why it matters
End each line with a colon `:`	Missing colon = `SyntaxError`
Indent the code inside (4 spaces)	The indent is how Python knows what belongs to the branch
The **first** True branch wins	Later branches are not even checked
`elif` and `else` are optional	Use as many `elif` as you need; at most one `else`

NOTEBOOK CELL 8

```
price  = 205.21
stock  = 0
rating = 3

is_in_stock    = stock > 0
is_affordable  = price < 300.0
is_recommended = rating >= 4

print(is_in_stock, is_affordable, is_recommended)

if is_in_stock and is_recommended:
    recommendation = "Great choice — in stock and highly rated!"
elif is_in_stock and not is_recommended:
    recommendation = "In stock but reviews are mixed."
else:
    recommendation = "Out of stock."

print(recommendation)
```

OUTPUT

```
False True False
Out of stock.
```

Here `stock = 0`, so `is_in_stock` is `False`. The first two branches need it to be True, so both are skipped and `else` runs.

18 Use case: route a customer query to the right agent

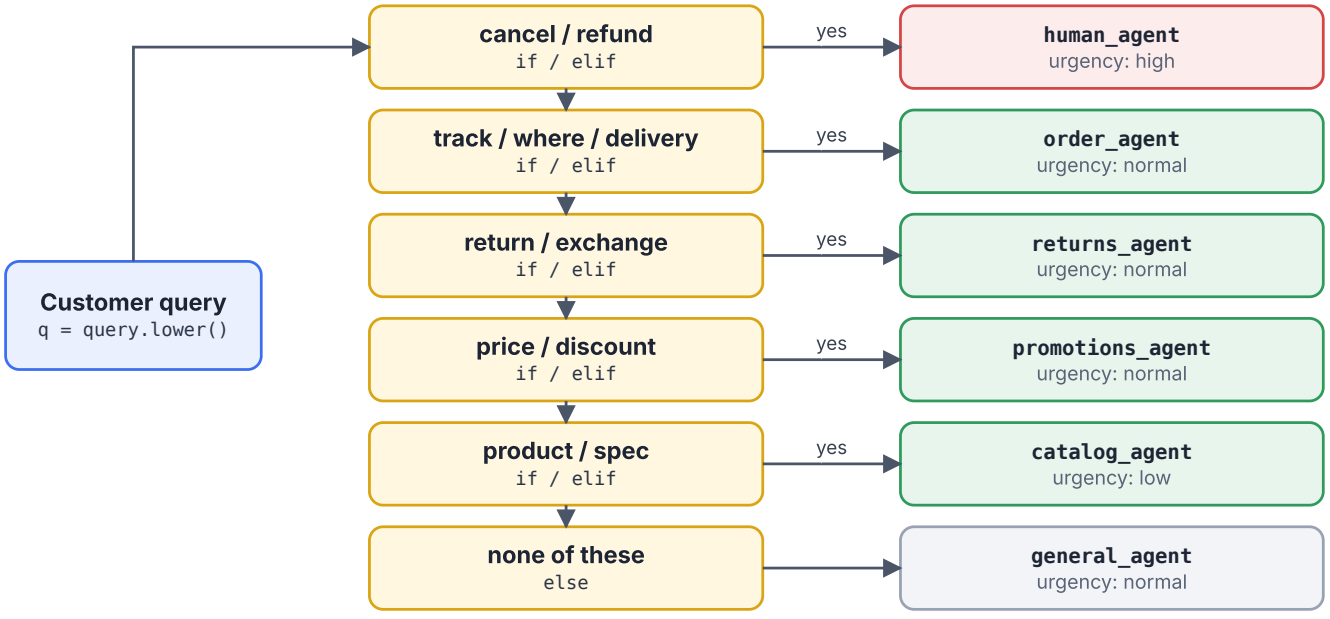


Figure 2.2 ShopSmart's router. The question is lower-cased once, then checked from the most urgent rule down.

A support assistant often has several specialist "agents". Before calling an LLM, simple rules decide which one should answer. `"Where is my order #3042?"` contains `where`, so it goes to `order_agent`.

NOTEBOOK CELL 11

```
customer_query = "Where is my order #3042?"
q = customer_query.lower()   # normalise once, use everywhere

if "cancel" in q or "refund" in q:
    agent, urgency = "human_agent", "high"
elif "track" in q or "where" in q or "delivery" in q:
    agent, urgency = "order_agent", "normal"
elif "return" in q or "exchange" in q:
    agent, urgency = "returns_agent", "normal"
elif "price" in q or "discount" in q:
    agent, urgency = "promotions_agent", "normal"
elif "product" in q or "spec" in q:
    agent, urgency = "catalog_agent", "low"
else:
    agent, urgency = "general_agent", "normal"

print(f"Query  : {customer_query}")
print(f"Agent  : {agent}")
print(f"Urgency: {urgency}")
```

OUTPUT

```
Query  : Where is my order #3042?
Agent  : order_agent
Urgency: normal
```

Order matters. Cancel and refund are checked first because they are urgent and need a human. `"Where can I cancel my order?"` contains both `where` and `cancel` — it goes to `human_agent` because that rule comes first.

Lower-case once (`q = customer_query.lower()`) and test `q` everywhere. Otherwise `"Cancel"` would not match `"cancel"` .

19 for loops: do something for every item

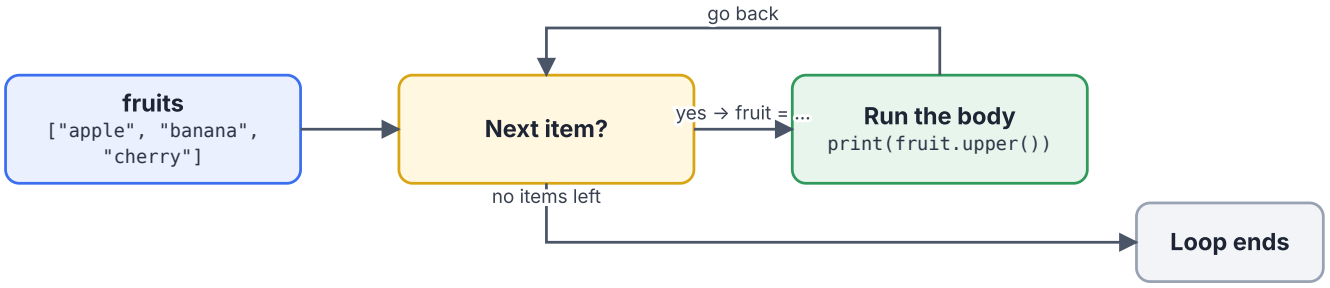


Figure 2.3 A for loop: take the next item, run the body, repeat until the collection is used up.

A `for` loop runs its body **once per item**. The loop variable (`fruit`) holds the current item. Code that is not indented runs once, after the loop finishes.

NOTEBOOK CELL 13

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    fruit_uppercase = fruit.upper()
    print(fruit_uppercase)
print("Hello World")
```

OUTPUT

```
APPLE
BANANA
CHERRY
Hello World
```

Iteration	fruit	Prints
1	`"apple"`	`APPLE`
2	`"banana"`	`BANANA`
3	`"cherry"`	`CHERRY`
—	list used up	loop ends → `Hello World`

Use	When
`for`	You have a collection, or know how many times to repeat
`while`	Repeat until something changes (e.g. retry until success) — see below

20 enumerate: the position and the item together

Sometimes you need the item **and** its position (to number lines, or to remember which query a result came from). `enumerate()` gives both. Add `start=1` to count from 1.

NOTEBOOK CELL 16

```
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits) :
    print(index, fruit)
```

OUTPUT

```
0 apple
1 banana
2 cherry
```

Note on the notebook: cell 18 has the same loop but prints only `f"{fruit}"`, so it shows the names without numbers. To get the `[0] apple` style from its comment, print `f"[{index}] {fruit}"`.

21 Loop + if: turn database values into customer messages

Put an `if` inside a loop to treat each item differently. Real data is messy, so the final `else` catches any status nobody planned for. `{status:12s}` pads the text to 12 characters so the arrows line up.

NOTEBOOK CELL 20

```
# Map database status values → customer-facing messages
# The else handles any unexpected value — real data is always messy
statuses = ["Delivered", "In Transit", "Cancelled", "Refunded", "Unknown"]

for status in statuses:
    if status == "Delivered":
        msg = "Your order has been delivered."
    elif status == "In Transit":
        msg = "Your order is on its way."
    elif status == "Processing":
        msg = "Your order is being prepared."
    elif status == "Cancelled":
        msg = "Your order has been cancelled."
    elif status == "Refunded":
        msg = "Your refund has been processed."
    else:
        msg = f"Status '{status}' — please contact support."
    print(f"{status:12s} → {msg}")
```

OUTPUT

```
Delivered    → Your order has been delivered.
In Transit   → Your order is on its way.
Cancelled    → Your order has been cancelled.
Refunded     → Your refund has been processed.
Unknown      → Status 'Unknown' — please contact support.
```

Padding in action — `"Prudhv"` has 6 letters, `:7s` adds one space:

NOTEBOOK CELL 21

```
aString = "Prudhv"
print(f"{aString:7s}--> done")
```

OUTPUT

```
Prudhv --> done
```

22 range: counting without a list

`range()` produces numbers on demand. Wrap it in `list()` to see them.

Form	Gives	Remember
`range(5)`	0 1 2 3 4	Starts at 0, stops **before** 5
`range(1, 6)`	1 2 3 4 5	Start, stop (stop not included)
`range(0, 10, 2)`	0 2 4 6 8	Start, stop, step
`range(1, 10, 5)`	1 6	Next would be 11 — past the stop

NOTEBOOK CELL 24

```
print(list(range(5)))           # [0, 1, 2, 3, 4]
print(list(range(1, 6)))        # [1, 2, 3, 4, 5]
print(list(range(0, 10, 2)))    # [0, 2, 4, 6, 8]
```

OUTPUT

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 2, 4, 6, 8]
```

23 Use case: process queries in batches



Figure 2.4 10 queries, batch size 3 → ceil(10 / 3) = 4 batches. The last batch is smaller.

LLM APIs limit how many requests you can send at once, so work is split into **batches**. `ceil()` rounds up, so 10 / 3 = 3.33 becomes 4 batches. Each batch is a slice `[start:end]`.

NOTEBOOK CELL 26

```
# Batch processing — split 9 queries into batches of 3
total_queries = 10
batch_size    = 3

aList = list(range(0,10))

from math import ceil

print(ceil(total_queries / batch_size))

for batch_num in range(ceil(total_queries / batch_size)):
    start = batch_num * batch_size
    end   = start + batch_size
    print(f"Batch {batch_num + 1}: queries {start} to {end - 1}")
    print(aList[start:end])
```

OUTPUT

```
4
Batch 1: queries 0 to 2
[0, 1, 2]
Batch 2: queries 3 to 5
[3, 4, 5]
Batch 3: queries 6 to 8
[6, 7, 8]
Batch 4: queries 9 to 11
[9]
```

Look at the last line: `queries 9 to 11` — but only query 9 exists. The slice is safe (Python stops at the end of the list), the printed label is not. Print from the batch itself instead:

```
from math import ceil

queries    = list(range(10))
batch_size = 3

for batch_num in range(ceil(len(queries) / batch_size)):
    start = batch_num * batch_size
    batch = queries[start:start + batch_size]
    print(f"Batch {batch_num + 1}: queries {batch[0]} to {batch[-1]} -> {batch}")
```

OUTPUT

```
Batch 1: queries 0 to 2 -> [0, 1, 2]
Batch 2: queries 3 to 5 -> [3, 4, 5]
Batch 3: queries 6 to 8 -> [6, 7, 8]
Batch 4: queries 9 to 9 -> [9]
```

24 Accumulator: collect results from a loop



Figure 2.5 The accumulator pattern: start empty, add one result per loop, use the full list after.

To keep results, create an empty list **before** the loop and `append` inside it. This test runs four sample questions through the router and stores each answer as a dict:

NOTEBOOK CELL 28

```
test_queries = [
    "Where is my order #3042?",
    "I want to cancel my purchase",
    "Do you have a discount code?",
    "What are the specs of the Classic Monitor?",
]

results = []   # accumulator — starts empty

for i, query in enumerate(test_queries):
    q = query.lower()
    if   "cancel" in q or "refund"  in q:
        agent = "human_agent"
    elif "where"  in q or "track"   in q:
        agent = "order_agent"
    elif "discount" in q or "price" in q:
        agent = "promotions_agent"
    elif "spec"   in q or "product" in q:
        agent = "catalog_agent"
    else:
        agent = "general_agent"

    results.append({"index": i, "query": query, "agent": agent})

for r in results:
    print(f"[{r['index']}] {r['query'][:38]:38s} → {r['agent']}")
```

OUTPUT

```
[0] Where is my order #3042?              → order_agent
[1] I want to cancel my purchase          → human_agent
[2] Do you have a discount code?          → promotions_agent
[3] What are the specs of the Classic Moni → catalog_agent
```

i	Query (lower-cased)	First rule that matches	agent
0	where is my order #3042?	`where`	`order_agent`
1	i want to cancel my purchase	`cancel` (checked first)	`human_agent`
2	do you have a discount code?	`discount`	`promotions_agent`
3	what are the specs of the classic monitor?	`spec`	`catalog_agent`

`{r['query'][:38]:38s}` cuts the text to 38 characters and pads it to 38 — that is why the last query shows `Classic Moni` .

25 while loops: repeat until a condition changes

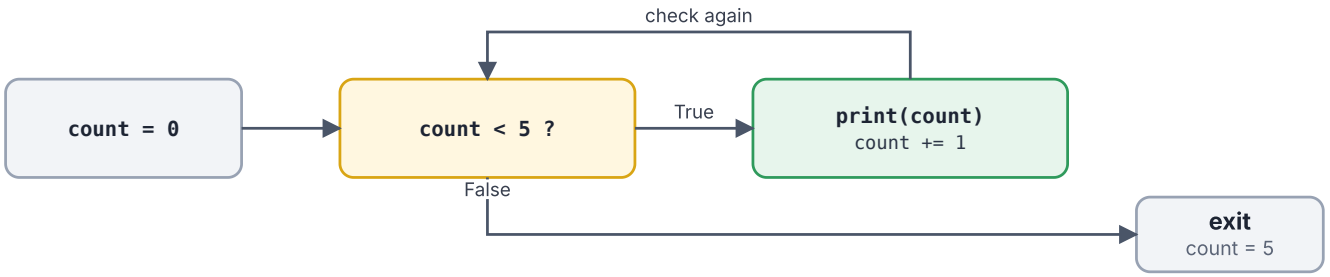


Figure 2.6 A while loop checks its condition before every round. Something inside must change it.

A `while` loop repeats **as long as** its condition is True. Use it when you don't know the number of rounds in advance — the classic AI example is "retry the API call until it succeeds or we give up".

NOTEBOOK CELL 31

```
count = 0

while count < 5:
    print(f"count = {count}")
    count += 1    # update the condition — without this: infinite loop

print(f"Loop ended. Final count = {count}")
```

OUTPUT

```
count = 0
count = 1
count = 2
count = 3
count = 4
Loop ended. Final count = 5
```

Forget `count += 1` and `count < 5` stays True forever: an **infinite loop**. In Jupyter, stop it with the ■ (interrupt) button.

The Day 02 summary mentions retries with exponential backoff, but the notebook has no code for it. The full retry example is in Day 05 of this guide ("Retry with exponential backoff").

26 break and continue

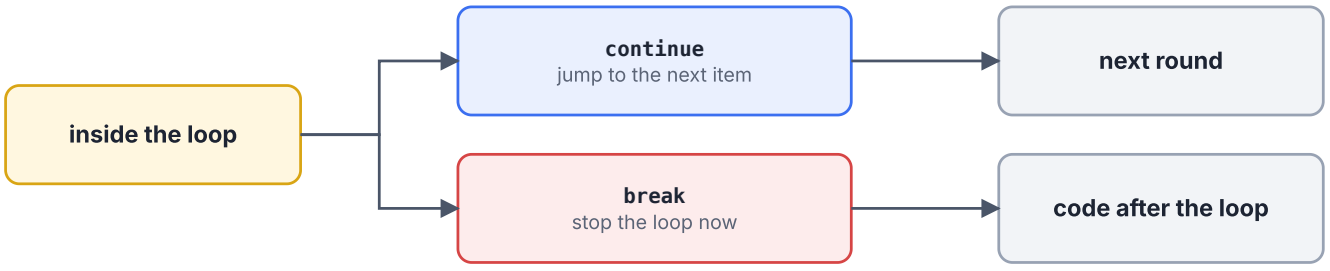


Figure 2.7 continue skips the rest of this round. break leaves the loop completely.

NOTEBOOK CELL 34

```
odds = []
for n in range(1, 10):
    if n % 2 == 0:
        continue        # even → skip this iteration, jump to n+1
    odds.append(n)
print("Odd numbers:", odds)  # [1, 3, 5, 7, 9]

for n in range(1, 10):
    if n > 5:
        print(f"Stopped at {n}")  # prints 6, then exits
        break
    print(n, end=' ')             # prints 1 2 3 4 5
print()
```

OUTPUT

```
Odd numbers: [1, 3, 5, 7, 9]
1 2 3 4 5 Stopped at 6
```

n	n % 2 == 0 ?	Action
1	No	append 1
2	Yes	continue (skip)
3	No	append 3
…	…	…
9	No	append 9

27 Use case: pick few-shot examples for a prompt

Few-shot prompting means putting a few good examples into the prompt so the LLM copies their style. We want at most **3** reviews with rating ≥ 4. `continue` skips weak reviews; `break` stops as soon as we have 3.

NOTEBOOK CELL 36

```
all_reviews = [
    {"review_id": 5001, "rating": 3, "title": "Its fine"},
    {"review_id": 5002, "rating": 5, "title": "Excellent!"},
    {"review_id": 5003, "rating": 4, "title": "Really good"},
    {"review_id": 5004, "rating": 2, "title": "Disappointed"},
    {"review_id": 5005, "rating": 5, "title": "Perfect product"},
    {"review_id": 5006, "rating": 4, "title": "Solid choice"},
]

good_reviews   = []   # accumulator — starts empty
max_to_collect = 3

for review in all_reviews:

    if review["rating"] < 4:
        continue                    # rating too low — skip to next iteration

    good_reviews.append(review)     # rating >= 4 — keep it

    if len(good_reviews) >= max_to_collect:
        break                       # collected enough — exit loop

# Print results
for r in good_reviews:
    print(f"  {r['review_id']}  {'★' * r['rating']}  {r['title']}")

print()
print("Selected titles:", [r["title"] for r in good_reviews])
print(f"review_id=5006 '{all_reviews[5]['title']}' was never reached — loop broke at iter 5")
```

OUTPUT

```
  5002  ★★★★★  Excellent!
  5003  ★★★★  Really good
  5005  ★★★★★  Perfect product

Selected titles: ['Excellent!', 'Really good', 'Perfect product']
review_id=5006 'Solid choice' was never reached — loop broke at iter 5
```

Round	review_id	rating	rating < 4 ?	What happens	good_reviews
1	5001	3	Yes	`continue` — skip	0
2	5002	5	No	append	1
3	5003	4	No	append	2
4	5004	2	Yes	`continue` — skip	2
5	5005	5	No	append → have 3 → `break`	3
—	5006	4	—	never checked	3

`'★' * r['rating']` repeats the star character: `'★' * 4` → `★★★★`. Multiplying a string repeats it.

28 List comprehensions: build a list in one line



Figure 2.8 A list comprehension is a for loop + if + append, folded into one line.

Pattern: `[expression for item in collection if condition]`. Read it as "give me `expression` for each `item` in `collection`, keeping only those where `condition` is True".

NOTEBOOK CELL 38

```
good_reviews   = []   # accumulator — starts empty
for review in all_reviews:
    if review["rating"] > 4:
        good_reviews.append(review)
    else:
        continue

# List comprehension
good_reviews_list_comp = [review for review in all_reviews if review["rating"] > 4]
print(good_reviews_list_comp)
print(good_reviews)
```

OUTPUT

```
[{'review_id': 5002, 'rating': 5, 'title': 'Excellent!'}, {'review_id': 5005, 'rating': 5, 'title':
'Perfect product'}]
[{'review_id': 5002, 'rating': 5, 'title': 'Excellent!'}, {'review_id': 5005, 'rating': 5, 'title':
'Perfect product'}]
```

Note `> 4` here (only 5-star reviews), while the few-shot loop used `>= 4`. More small examples:

NOTEBOOK CELL 41

```
squares = [n * n  for n in range(1, 6)]

evens   = [n  for n in range(1, 11)  if n % 2 == 0]

upper   = [w.upper()  for w in ["hello", "world", "python"]]

print("Squares:", squares)
print("Evens:  ", evens)
print("Upper:  ", upper)
```

OUTPUT

```
Squares: [1, 4, 9, 16, 25]
Evens:   [2, 4, 6, 8, 10]
Upper:   ['HELLO', 'WORLD', 'PYTHON']
```

Use case: filter products and build prompt lines

Keep only in-stock products, and turn affordable ones (price < 300) into lines ready to paste into a prompt:

NOTEBOOK CELL 43

```
products = [
    {"name": "Classic Monitor",   "price": 205.21, "stock": 238},
    {"name": "Ultimate Perfume",  "price": 568.17, "stock": 10},
    {"name": "Budget Headphones", "price": 29.99,  "stock": 0},
    {"name": "Yoga Mat Pro",      "price": 45.00,  "stock": 150},
    {"name": "Luxury Cream",      "price": 89.00,  "stock": 0},
]

in_stock = [p for p in products  if p["stock"] > 0]
print("In stock:", [p["name"] for p in in_stock])

prompt_lines = [
    f"- {p['name']} (${p['price']:.2f})"
    for p in products
    if p["stock"] > 0 and p["price"] < 300
]
print("Prompt lines:")
for line in prompt_lines:
    print(" ", line)
```

OUTPUT

```
In stock: ['Classic Monitor', 'Ultimate Perfume', 'Yoga Mat Pro']
Prompt lines:
  - Classic Monitor ($205.21)
  - Yoga Mat Pro ($45.00)
```

Product	stock > 0	price < 300	In prompt?
Classic Monitor	238 ✓	205.21 ✓	Yes
Ultimate Perfume	10 ✓	568.17 ×	No
Budget Headphones	0 ×	—	No
Yoga Mat Pro	150 ✓	45.00 ✓	Yes
Luxury Cream	0 ×	—	No

Use a comprehension for simple filter-and-transform. If you need `break`, several steps or printing, write a normal loop — it is easier to read.

29 Dict comprehensions

Same idea, but with `key: value` in curly braces: `{key: value for item in collection}`.

NOTEBOOK CELL 46

```
squares_dict = {n: n*n        for n in range(1, 6)}

word_lengths  = {w: len(w)    for w in ["apple", "banana", "cherry"]}

print(squares_dict)            # {1:1, 2:4, 3:9, 4:16, 5:25}
print(squares_dict[3])         # 9 — instant O(1) lookup
print(word_lengths)            # {'apple': 5, 'banana': 6, 'cherry': 6}
```

OUTPUT

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
9
{'apple': 5, 'banana': 6, 'cherry': 6}
```

Why build a dict? Looking up `squares_dict[3]` is instant, however big the dict. Finding the same value in a list means checking item after item.

30 Day 02 recap

Concept	Remember
Condition	A True/False question; combine with `and`, `or`, `not`; `in` checks inside text or collections
if / elif / else	Top to bottom; first True branch wins; put the urgent rules first
for	Once per item; the unindented line after runs once at the end
enumerate	`for i, item in enumerate(items)` — position + item
range	`range(start, stop, step)` — stop is not included
Batching	`ceil(total / size)` batches; slice `[start:start + size]`
Accumulator	`results = []` before the loop, `append` inside
while	Repeat while True; change something inside or it never ends
continue / break	Skip this round / leave the loop
Comprehensions	`[x for x in xs if …]` and `{k: v for …}`

DAY 03

Functions & Data Structures

Write code once and reuse it: functions with inputs, defaults and flexible arguments. Then the everyday methods of lists and dicts — the tools you use to build chat history and look products up by name.

Notebook: **03_Functions_DataStructures.ipynb**

def & call | Parameters & return | Discount example | Defaults | *args | **kwargs | List methods | Message history | Dict methods | Index by name | Sets | Tuples & zip

31 Functions: write once, reuse anywhere

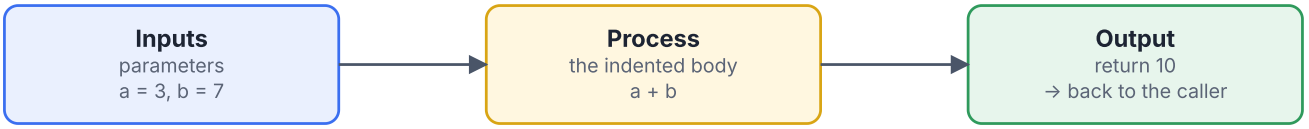


Figure 3.1 Every function in this notebook follows the same shape: inputs go in, work happens, one result comes back.

A **function** is a named block of code. `def` **defines** it (nothing runs yet). Writing its name with brackets **calls** it — then the body runs. Call it as many times as you like.

NOTEBOOK CELL 2

```
def greet():
    print("Hello! Welcome.")

greet()
greet()
```

OUTPUT

```
Hello! Welcome.
Hello! Welcome.
```

Parameters and return

Parameters are the inputs, written inside the brackets. `return` hands a result back to the code that called the function, so it can be stored in a variable or used in another expression.

NOTEBOOK CELL 4

```
def add(a, b):
    return a + b

result = add(3, 7)
print(result)
print(add(10, 20))
```

OUTPUT

```
10
30
```

	`print()`	`return`
What it does	Shows a value on the screen	Sends a value back to the caller
Can the caller use the value?	No — it is only displayed	Yes — `result = add(3, 7)`
Function without it gives	—	`None`

A function that only prints returns `None`. If `result = my_function()` gives you `None`, you probably forgot `return`.

32 Use case: calculate a discount

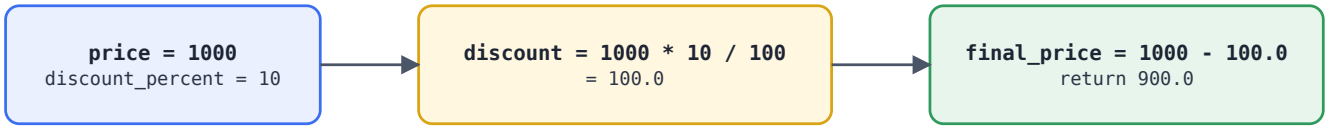


Figure 3.2 apply_discount(1000, 10) step by step.

Same input → process → return shape, now with real meaning: a price and a discount percentage go in, the final price comes out. The result is a float because `/` always gives a float.

NOTEBOOK CELL 6

```
def apply_discount(price, discount_percent):
    discount = price * discount_percent / 100
    final_price = price - discount
    return final_price

# Try different prices and discounts
print(apply_discount(1000, 10))  # 10% off 1000 → 900.0
print(apply_discount(2000, 15))  # 15% off 2000 → 1700.0
print(apply_discount(500, 20))   # 20% off 500 → 400.0
```

OUTPUT

```
900.0
1700.0
400.0
```

Call it with one value — `apply_discount(1000)` — and Python stops with `TypeError: apply_discount() missing 1 required positional argument: 'discount_percent'`. (The original notebook still shows this old error under cell 6.) Every parameter without a default must be given. The next section fixes that.

33 Default parameter values

Give a parameter a value in the `def` line and it becomes **optional**. Leave it out → Python uses the default. Pass it → your value wins.

NOTEBOOK CELL 8

```
def power(base, exponent=2):
    return base ** exponent

print(power(3))       # Uses exponent=2 → 9
print(power(3, 3))    # Uses exponent=3 → 27
```

OUTPUT

```
9
27
```

```
def apply_discount(price, discount_percent=10):     # practice task 2 from the notebook
    return price - price * discount_percent / 100

apply_discount(1000)       # 900.0  — uses the default 10
apply_discount(1000, 25)   # 750.0
```

Parameters with defaults must come **after** the ones without: `def f(price, discount=10)` is fine, `def f(discount=10, price)` is a `SyntaxError`.

34 *args: any number of positional values

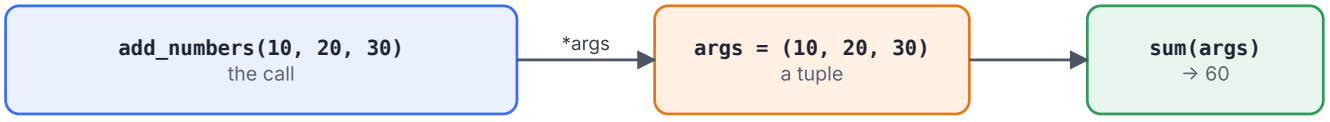


Figure 3.3 *args gathers every extra positional value into one tuple.

When you don't know how many values will be passed, put `*` before a parameter name. Python collects all of them into a **tuple**. `args` is only a convention — the `*` is what matters (cell 10 uses `*numbers`).

NOTEBOOK CELL 10

```
def rand_function(*numbers):
    print(type(numbers))
    sum = 0
    for number in numbers:
        sum += number
    return sum

print(rand_function(1,2,3,4,5))
```

OUTPUT

```
<class 'tuple'>
15
```

Cell 10 names a variable `sum`. That hides Python's built-in `sum()` function for the rest of the notebook. It works here, but pick a name like `total` instead.

NOTEBOOK CELL 11

```
def add_numbers(*args):
    print("Received:", args)
    return sum(args)

print("Two numbers:", add_numbers(10, 20))
print("Three numbers:", add_numbers(10, 20, 30))
print("Five numbers:", add_numbers(10, 20, 30, 40, 50))
```

OUTPUT

```
Received: (10, 20)
Two numbers: 30
Received: (10, 20, 30)
Three numbers: 60
Received: (10, 20, 30, 40, 50)
Five numbers: 150
```

35 **kwargs: any number of named settings



Figure 3.4 **kwargs gathers every extra name=value pair into one dict.

Two stars collect **named** arguments (`name=value`) into a **dict**. Inside, read them with `.get()` so a missing setting falls back to a default — the second call has no temperature, so it uses 0.7.

NOTEBOOK CELL 14

```
def show_config(**kwargs):
    model = kwargs.get("model", "Default Model")
    temperature = kwargs.get("temperature", 0.7)

    print("All settings:", kwargs)
    print(f"Model: {model}")
    print(f"Temperature: {temperature}")

show_config(model="Claude", temperature=0.2)
show_config(model="GPT")
```

OUTPUT

```
All settings: {'model': 'Claude', 'temperature': 0.2}
Model: Claude
Temperature: 0.2
All settings: {'model': 'GPT'}
Model: GPT
Temperature: 0.7
```

	`*args`	`**kwargs`
Collects	Extra positional values	Extra `name=value` values
Stored as	tuple	dict
Example call	`func(10, 20)`	`func(model="Claude")`

Both together

Plain values go to `args`, named ones to `kwargs` — a handy shape for an LLM request helper:

NOTEBOOK CELL 18

```
def process_request(*args, **kwargs):
    print("Inputs:", args)
    print("Configuration:", kwargs)

process_request(
    "Where is my order?",
    "customer_123",
    model="Claude",
    temperature=0.2
)
```

OUTPUT

```
Inputs: ('Where is my order?', 'customer_123')
Configuration: {'model': 'Claude', 'temperature': 0.2}
```

36 Lists and their everyday methods

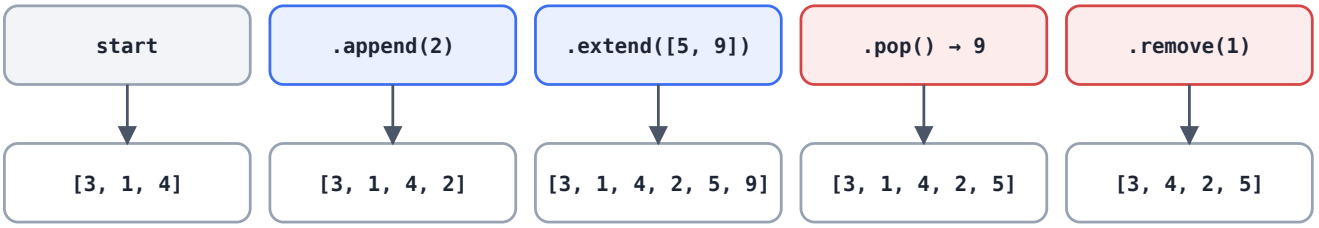


Figure 3.5 The same list, changed by four methods in turn.

NOTEBOOK CELL 21

```
products = ["Classic Monitor", "Ultimate Perfume", "Yoga Mat Pro"]
print(products)
print(products[0])    # First item
print(len(products))  # Number of items
```

OUTPUT

```
['Classic Monitor', 'Ultimate Perfume', 'Yoga Mat Pro']
Classic Monitor
3
```

Method	Does	Works by
`append(x)`	Adds **one** item to the end	—
`extend(items)`	Adds **each** item of another collection	—
`pop()` / `pop(i)`	Removes and **returns** the last item (or item `i`)	position
`remove(x)`	Removes the first item equal to `x`	value

NOTEBOOK CELL 23

```
numbers = [3, 1, 4]

numbers.append(2)
print("After append:", numbers)

numbers.extend([5, 9])
print("After extend:", numbers)

last_number = numbers.pop()
print("Removed by pop:", last_number)
print("After pop:", numbers)

numbers.remove(1)
print("After remove(1):", numbers)
```

OUTPUT

```
After append: [3, 1, 4, 2]
After extend: [3, 1, 4, 2, 5, 9]
Removed by pop: 9
After pop: [3, 1, 4, 2, 5]
After remove(1): [3, 4, 2, 5]
```

`numbers.append([5, 9])` would add **one** item — the whole list — giving `[3, 1, 4, 2, [5, 9]]`. Use `extend` to add the items one by one.

37 Use case: build chat message history

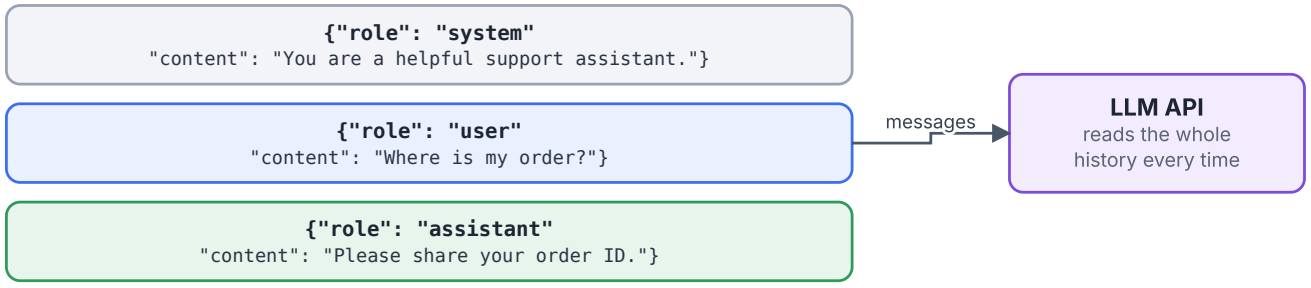


Figure 3.6 A chat is a list of message dicts. Every turn appends one more; the whole list is sent to the LLM.

LLM chat APIs take a **list of dicts**, each with a `role` and `content`. The list keeps the order of the conversation, and `append` adds each new turn:

NOTEBOOK CELL 25

```
messages = [
    {"role": "system", "content": "You are a helpful support assistant."}
]

messages.append({"role": "user", "content": "Where is my order?"})
messages.append({"role": "assistant", "content": "Please share your order ID."})

print(messages)
```

OUTPUT

```
[{'role': 'system', 'content': 'You are a helpful support assistant.'}, {'role': 'user', 'content': 'Where is my order?'}, {'role': 'assistant', 'content': 'Please share your order ID.'}]
```

38 Dictionaries: reading values safely

NOTEBOOK CELL 27

```
product = {"name": "Classic Monitor", "price": 205.21, "stock": 238}

print(product["name"])
print(product["price"])
```

OUTPUT

```
Classic Monitor
205.21
```

Way	Key missing →
`product["stock"]`	`KeyError` — the program stops (Day 05 shows how to catch it)
`product.get("stock", 0)`	Returns the default `0` and carries on

NOTEBOOK CELL 29

```
product = {"name": "Classic Monitor", "price": 205.21}

print(product["name"])
print(product.get("stock", 0))          # Missing stock → 0
print(product.get("discount", "N/A"))   # Missing discount → N/A
```

OUTPUT

```
Classic Monitor
0
N/A
```

Use `[key]` when the key **must** be there (a missing one is a bug you want to see). Use `.get(key, default)` for optional fields — LLM replies and API data often leave fields out.

39 Dictionary methods

Method	Does
`keys()` / `values()` / `items()`	All keys / all values / all (key, value) pairs

Method	Does
`get(key, default)`	Read safely with a fallback
`update(other)`	Add new pairs and **overwrite** existing ones
`pop(key)`	Remove that key and return its value
`popitem()`	Remove and return the **last inserted** pair
`clear()`	Empty the dict

NOTEBOOK CELL 31

```
settings = {"model": "example-model", "temperature": 0.2}

print("Keys:", list(settings.keys()))
print("Values:", list(settings.values()))
print("Pairs:", list(settings.items()))
print("Missing setting:", settings.get("region", "not set"))

settings.update({"temperature": 0.0, "max_tokens": 512})
print("After update:", settings)

removed_value = settings.pop("max_tokens")
print("Removed value:", removed_value)
print("After pop:", settings)

removed_pair = settings.popitem()
print("Removed last pair:", removed_pair)
print("After popitem:", settings)

settings.clear()
print("After clear:", settings)
```

OUTPUT

```
Keys: ['model', 'temperature']
Values: ['example-model', 0.2]
Pairs: [('model', 'example-model'), ('temperature', 0.2)]
Missing setting: not set
After update: {'model': 'example-model', 'temperature': 0.0, 'max_tokens': 512}
Removed value: 512
After pop: {'model': 'example-model', 'temperature': 0.0}
Removed last pair: ('temperature', 0.0)
After popitem: {'model': 'example-model'}
After clear: {}
```

After	settings
start	`{'model': 'example-model', 'temperature': 0.2}`
`update(...)`	temperature overwritten to `0.0`, `max_tokens: 512` added
`pop('max_tokens')`	returns `512`, key gone
`popitem()`	returns `('temperature', 0.0)` — the last pair
`clear()`	`{}`

40 Use case: index products by name

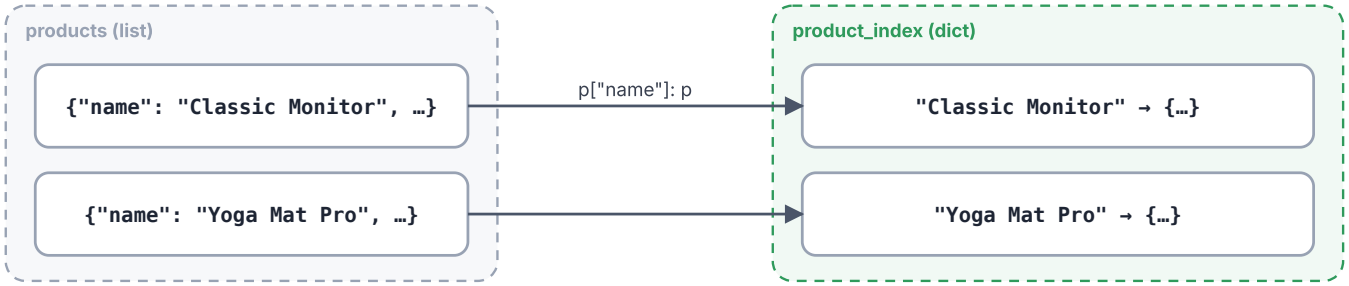


Figure 3.7 Turn a list into a dict once; then every lookup by name is instant.

A dict comprehension (Day 02) uses each product's name as the key and the full product as the value. After that, `product_index["Classic Monitor"]` jumps straight to the record — no loop needed.

NOTEBOOK CELL 33

```
products = [
    {"name": "Classic Monitor", "price": 205.21, "stock": 238},
    {"name": "Yoga Mat Pro", "price": 45.00, "stock": 150},
]

product_index = {p["name"]: p for p in products}

classic = product_index["Classic Monitor"]
print(f"Price: {classic['price']}  Stock: {classic['stock']}")
```

OUTPUT

```
Price: 205.21  Stock: 238
```

41 Sets: unique values

A quick refresher from Day 01: duplicates disappear, and sets can be combined.

NOTEBOOK CELL 35

```
a = {1, 2, 3, 3, 4}
b = {3, 4, 5, 6}

print("a:", a)                    # Duplicate 3 appears only once
print("Union:", a | b)            # Items in either set
print("Intersection:", a & b)     # Items in both sets
print("Difference:", a - b)       # Items in a but not b
```

OUTPUT

```
a: {1, 2, 3, 4}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference: {1, 2}
```

Operator	Gives	Here
`a | b`	In either set (union)	`{1, 2, 3, 4, 5, 6}`
`a & b`	In both (intersection)	`{3, 4}`

Operator	Gives	Here
`a - b`	In `a` but not in `b`	`{1, 2}`

42 Tuples and zip()

A function that returns two values separated by a comma really returns one **tuple**. Read the parts by position, or unpack them: `low, high = min_max([3, 1, 9, 4])`.

NOTEBOOK CELL 37

```
def min_max(numbers):
    return min(numbers), max(numbers)   # Returns a tuple

result = min_max([3, 1, 9, 4])
print(result)
print("Minimum:", result[0])
print("Maximum:", result[1])
```

OUTPUT

```
(1, 9)
Minimum: 1
Maximum: 9
```

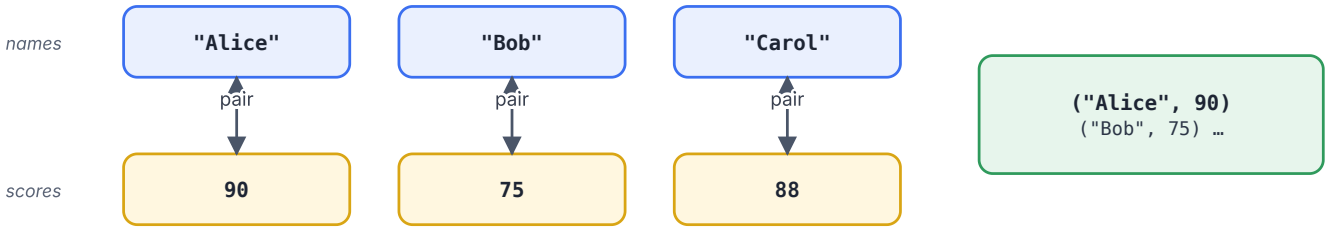


Figure 3.8 zip() walks two lists side by side and pairs items by position.

`zip()` pairs items from two (or more) lists by position. Each pair is a tuple, unpacked straight into `name, score`:

NOTEBOOK CELL 38

```
names = ["Alice", "Bob", "Carol"]
scores = [90, 75, 88]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

OUTPUT

```
Alice: 90
Bob: 75
Carol: 88
```

If the lists have different lengths, `zip()` stops at the shorter one — extra items are silently ignored.

43 Day 03 recap and practice

Concept	Remember
`def` / call	Define once, call by name with `()`
Parameters / `return`	Inputs in; one result back. No `return` → `None`
Default parameter	`def f(price, discount=10)` — optional, defaults go last
`*args` / `**kwargs`	Extra positional → tuple / extra named → dict
List methods	`append`, `extend`, `pop` (position), `remove` (value)
Message history	A list of `{"role", "content"}` dicts, appended each turn
Dict methods	`get`, `update`, `pop`, `popitem`, `keys/values/items`, `clear`
Index by name	`{p["name"]: p for p in products}`
`zip()`	Pairs items by position; stops at the shorter list

Practice (from the notebook)

1. Write a function that converts Celsius to Fahrenheit and returns the result.
2. Update `apply_discount()` to accept a default discount of `10`.
3. Write a function using `*args` that returns the largest number.
4. Write a function using `**kwargs` that prints each setting.
5. Create a list of three products and append one more product.
6. Use a dictionary's `get()` method to handle a missing key.
7. Convert a list with duplicate values into a set.
8. Use `zip()` to pair product names with prices.

DAY 04

File I/O, JSON & CSV

Programs need data that lives outside the code: prompts in text files, settings in JSON, customers in CSV, and logs of every LLM call. Today: move data between Python, strings and files — safely.

Notebook: **04_File_IO_JSON.ipynb**

What JSON is | loads / dumps | load / dump | with open() | Prompt from file | JSONL log | Fenced LLM JSON | csv.DictReader | Other CSV shapes | Mini AI workflow

44 What JSON is

JSON is a text format for structured data. APIs, LLM replies and config files all use it. It looks like a Python dict, but it is **text** with slightly different spelling:

JSON	Python
object `{...}`	`dict`
array `[...]`	`list`
`true` / `false`	`True` / `False`
`null`	`None`
strings in **double** quotes only	single or double quotes

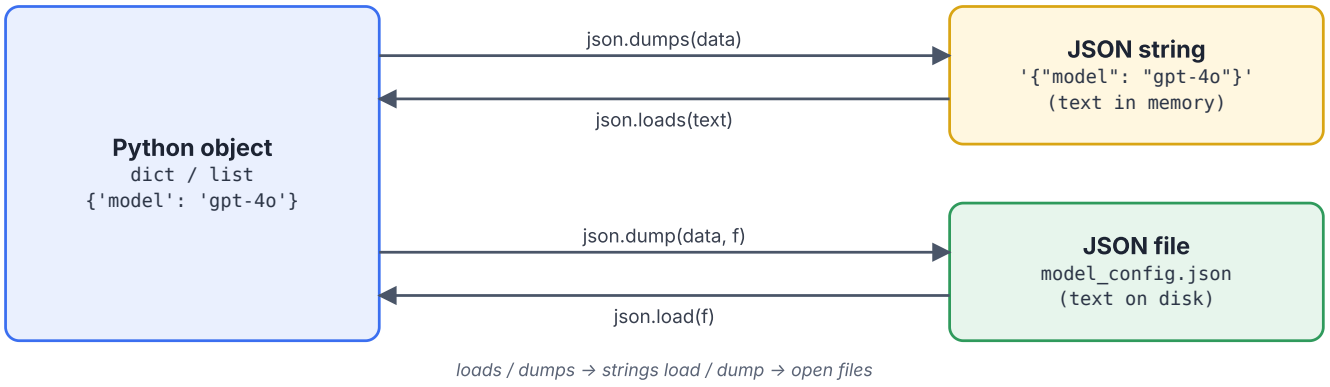


Figure 4.1 Three different things, four functions. The "s" means string.

Function	From → to	Use it when
`json.loads(text)`	JSON string → Python	Reading an API or LLM reply
`json.dumps(data)`	Python → JSON string	Logging, sending data to an API
`json.dump(data, f)`	Python → JSON file	Saving settings or results
`json.load(f)`	JSON file → Python	Reading a saved `.json` file

45 json.loads() and json.dumps(): strings

`loads` turns JSON **text** into a dict you can index. Notice the type changes from `str` to `dict`:

NOTEBOOK CELL 9

```
# ── json.loads() — string → dict ──────────────────────────
raw = '{"category": "TRACK_ORDER", "confidence": "high"}'

print('Before:', type(raw), raw)
parsed = json.loads(raw)
print('After :', type(parsed), parsed)
print('Access:', parsed['category'])
```

OUTPUT

```
Before: <class 'str'> {"category": "TRACK_ORDER", "confidence": "high"}
After : <class 'dict'> {'category': 'TRACK_ORDER', 'confidence': 'high'}
Access: TRACK_ORDER
```

`dumps` goes the other way. Without `indent` you get one compact line (good for logs); with `indent=2` a readable layout (good for files people open):

NOTEBOOK CELL 11

```
# ── json.dumps() — dict → string ──────────────────────────
config = {'model': 'gpt-4o', 'max_tokens': 1024, 'temperature': 0.2}

compact = json.dumps(config)           # single line — for logs
pretty  = json.dumps(config, indent=2) # indented — for .json files

print('Compact:', compact)
print('Type   :', type(compact))   # str
print()
print('Pretty:')
print(pretty)
```

OUTPUT

```
Compact: {"model": "gpt-4o", "max_tokens": 1024, "temperature": 0.2}
Type   : <class 'str'>

Pretty:
{
  "model": "gpt-4o",
  "max_tokens": 1024,
  "temperature": 0.2
}
```

46 json.dump() and json.load(): files

The file versions take an **open file** as the second input. First save the config, then read it back:

NOTEBOOK CELL 13

```
# json.dump  → write a dict directly to an open file
with open('model_config.json', 'w', encoding='utf-8') as f:
    json.dump(config, f, indent=2)
```

NOTEBOOK CELL 15

```
# ── json.load() and json.dump() — file versions ──────────────

# json.load  → read a dict directly from an open file

with open('model_config.json', 'r', encoding='utf-8') as f:
    loaded = json.load(f)

print('Loaded back:', loaded)
print('Type       :', type(loaded).__name__)   # dict
```

OUTPUT

```
Loaded back: {'model': 'gpt-4o', 'max_tokens': 1024, 'temperature': 0.2}
Type       : dict
```

Round trip: dict → file → dict. `loaded` equals `config`. This is how apps keep settings between runs.

47 with open(): safe file handling

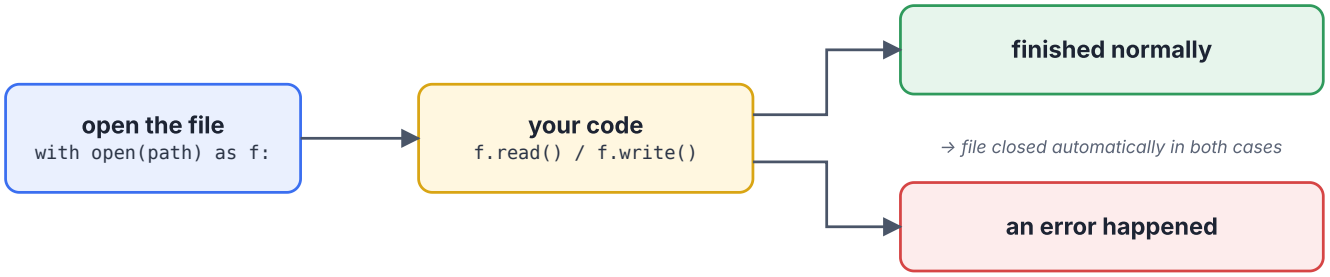


Figure 4.2 with open(...) closes the file on the way out — on success and on error.

An open file must be closed, or data can be lost and the file stays locked. With a plain `open()` an error in the middle skips your `close()`. The `with` block closes it for you, always.

```
# risky                                  # safe
f = open('file.txt')                     with open('file.txt') as f:
data = f.read()   # error here →           data = f.read()
f.close()         # …never runs          # closed here, error or not
```

Mode	Meaning	If the file…
`'r'`	Read	is missing → `FileNotFoundError`
`'w'`	Write	exists → **wiped and overwritten**; missing → created
`'a'`	Append	exists → add to the end; missing → created

Always pass `encoding='utf-8'`. Windows otherwise uses a different default and names like "José" or the ₹ sign can turn into garbage.

48 Use case: load a system prompt from a file



Figure 4.3 Keep prompts in files: the team edits text, the code stays the same.

First write a sample prompt file (mode `'w'`). `\n` is the "new line" character:

NOTEBOOK CELL 20

```
# Write a prompt to a file — simulates prompts/system_prompt.txt
with open('sample_prompt.txt', 'w', encoding='utf-8') as f:
    f.write('## Role\n')
    f.write('You are a customer support assistant for ShopSmart.\n')
    f.write('\n')
    f.write('## Task\n')
    f.write('Answer using ONLY the information provided.')
```

The function reads the whole file into one string. `.strip()` removes the extra newline editors add at the end:

NOTEBOOK CELL 19

```
def load_system_prompt(filepath='sample_prompt.txt'):
    """
    Load a system prompt from a .txt file.
    Keeping prompts in files means the team can edit them without touching Python.
    """
    # 'r' = read mode  |  encoding='utf-8' = always specify
    # with block closes the file automatically — even if f.read() raises an error
    with open(filepath, 'r', encoding='utf-8') as f:
        content = f.read()   # loads the entire file as one string

    # .strip() removes the trailing newline that text editors always add
    return content.strip()
```

NOTEBOOK CELL 21

```
# Now call the function — reads the file back
prompt = load_system_prompt('sample_prompt.txt')
print(f'Loaded {len(prompt)} characters:')
print(prompt)
```

OUTPUT

```
Loaded 112 characters:
## Role
You are a customer support assistant for ShopSmart.

## Task
Answer using ONLY the information provided.
```

49 Use case: log every LLM call (JSONL)

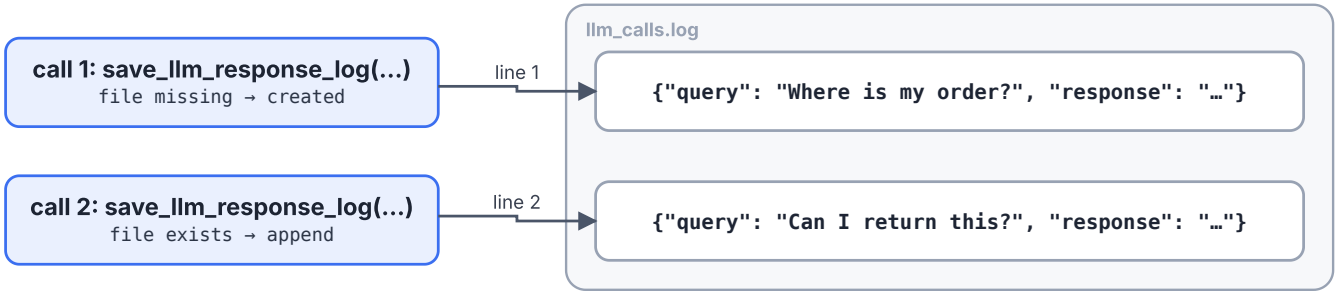


Figure 4.4 Append mode adds one JSON line per call. Earlier lines are never touched.

JSONL (JSON Lines) means one JSON object per line. It is the standard way to log LLM calls: easy to append, easy to read back line by line. The function uses `json.dumps()` + mode `'a'`:

NOTEBOOK CELL 23

```
def save_llm_response_log(query, response, log_file='llm_calls.log'):
    """
    Append one LLM call to a JSONL log file.
    JSONL = JSON Lines: one JSON object per line.
    """
    with open(log_file, 'a', encoding='utf-8') as f:
        # json.dumps() converts a Python dict → JSON string
        entry = json.dumps({'query': query, 'response': response})
        f.write(entry + '\n')   # one entry per line = JSONL format
```

```
save_llm_response_log('Where is my order?', 'Your order ORD-3042 is In Transit.')
save_llm_response_log('Can I return this?', 'Returns accepted within 30 days.')
```

Read it back: each line is a JSON string, so `json.loads()` turns it into a dict again:

NOTEBOOK CELL 27

```
print('Log contents:')
with open('llm_calls.log', 'r', encoding='utf-8') as f:
    for i, line in enumerate(f):
        record = json.loads(line.strip())
        print(f'  [{i+1}] Q: {record["query"]}')
        print(f'       A: {record["response"]}')
```

OUTPUT

```
Log contents:
  [1] Q: Where is my order?
       A: Your order ORD-3042 is In Transit.
  [2] Q: Can I return this?
       A: Returns accepted within 30 days.
```

Append mode never removes anything. Run the two calls again and the log has 4 lines — that is why the notebook's final step later reports 3 entries in a fresh run, and more if you re-run cells.

50 Use case: parse JSON returned by an LLM

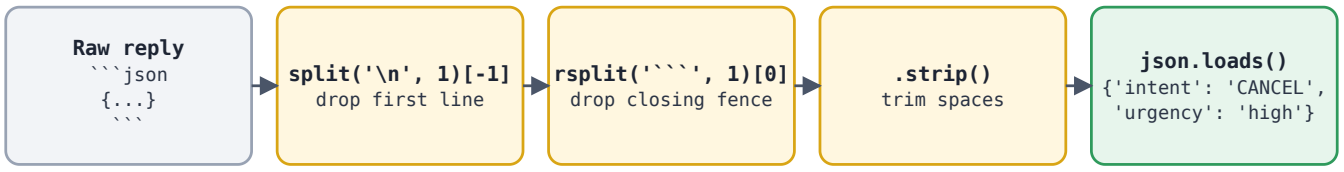


Figure 4.5 Cleaning an LLM reply that came wrapped in a Markdown code fence.

You ask the model for JSON, and it often wraps the JSON in a Markdown code fence — a first line of three backticks plus `json`, and a last line of three backticks. `json.loads()` would fail on the backticks, so remove the fence first:

NOTEBOOK CELL 30

```
def parse_llm_json_response(raw_response):
    """
    Parse a JSON string from the LLM into a Python dict.
    Also handles responses wrapped in markdown fences:
        ```json
        {"key": "value"}
        ```
    """
    # Step 1: remove leading/trailing whitespace
    cleaned = raw_response.strip()

    # Step 2: strip markdown fences if present
    # LLMs often add ```json ... ``` even when told not to
    if cleaned.startswith('```'):
        # split('\n', 1) → split on first newline only
        # [-1] takes everything after the opening fence line
        cleaned = cleaned.split('\n', 1)[-1]
        # rsplit('```', 1) → split on last ``` only
        # [0] takes everything before the closing fence
        cleaned = cleaned.rsplit('```', 1)[0].strip()

    # Step 3: convert JSON string → Python dict
    return json.loads(cleaned)
```

NOTEBOOK CELL 31

```
# ── Demo 1: plain JSON string ──────────────────────────────
raw = '{"category": "TRACK_ORDER", "confidence": "high"}'

print('Before parse — type:', type(raw).__name__)   # str
parsed = parse_llm_json_response(raw)
print('After parse  — type:', type(parsed).__name__) # dict
print('category  :', parsed['category'])
print('confidence:', parsed['confidence'])
```

OUTPUT

```
Before parse — type: str
After parse  — type: dict
category  : TRACK_ORDER
confidence: high
```

NOTEBOOK CELL 32

```
# ── Demo 2: LLM wrapped response in markdown fences ────────
# This is very common — LLMs add ```json even when told not to
fenced = '```json\n{"intent": "CANCEL", "urgency": "high"}\n```'

print('Raw response from LLM:')
print(fenced)
print()

parsed2 = parse_llm_json_response(fenced)
print('After parse:', parsed2)
print('intent    :', parsed2['intent'])
```

OUTPUT

```
Raw response from LLM:
```json
{"intent": "CANCEL", "urgency": "high"}
```

After parse: {'intent': 'CANCEL', 'urgency': 'high'}
intent   : CANCEL
```

If the text still isn't valid JSON, `json.loads()` raises `json.JSONDecodeError`. Day 05 shows how to catch it so one bad reply doesn't crash the program.

51 Read CSV rows with csv.DictReader

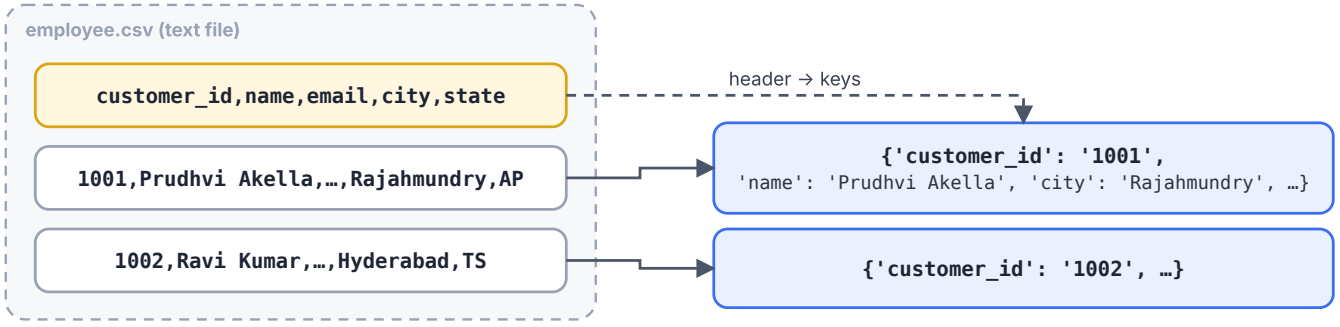


Figure 4.6 csv.DictReader uses the first row as keys and turns every other row into a dict.

A CSV file is a table saved as text: one row per line, values separated by commas. `csv.DictReader` gives you each row as a dict — the same shape as a database row on Day 01.

The notebook reads `employee.csv`, which is not included with it. Create it next to the notebook with this content:

```
customer_id,name,email,city,state
1001,Prudhvi Akella,prudhvi@example.com,Rajahmundry,AP
1002,Ravi Kumar,ravi@example.com,Hyderabad,TS
1003,Ananya Sharma,ananya@example.com,Bangalore,KA
```

NOTEBOOK CELL 36

```
import csv
# sample_csv simulates what customers.csv looks like on disk
# io.StringIO wraps a string so csv.DictReader treats it like an open file
with open('employee.csv', 'r', encoding='utf-8') as f:
    reader    = csv.DictReader(f)
    customers = list(reader)
    print(customers)
```

OUTPUT

```
[{'customer_id': '1001', 'name': 'Prudhvi Akella', 'email': 'prudhvi@example.com', 'city':
'Rajahmundry', 'state': 'AP'}, {'customer_id': '1002', 'name': 'Ravi Kumar', 'email':
'ravi@example.com', 'city': 'Hyderabad', 'state': 'TS'}, {'customer_id': '1003', 'name': 'Ananya
Sharma', 'email': 'ananya@example.com', 'city': 'Bangalore', 'state': 'KA'}]
```

The CSV type trap

Every value from a CSV is **text**, even `1001`. Text compares letter by letter, so `'1001' > '200'` is False (because `'1'` comes before `'2'`). Convert with `int()` or `float()` before comparing or doing maths:

NOTEBOOK CELL 37

```
for c in customers:
    print(f"  {c['customer_id']}  {c['name']:20s}  {c['city']}, {c['state']}")

print()
print('--- CSV type trap ---')
print('customer_id type    :', type(customers[0]['customer_id']).__name__, '← str!')
print("String compare trap : '1001' > '200' =", '1001' > '200', '← WRONG (alphabetic)')
print('Correct int compare : 1001  > 200  =',   1001  > 200)
print('Fix                 :', int(customers[0]['customer_id']))
```

OUTPUT

```
  1001  Prudhvi Akella        Rajahmundry, AP
  1002  Ravi Kumar           Hyderabad, TS
  1003  Ananya Sharma        Bangalore, KA

--- CSV type trap ---
customer_id type    : str ← str!
String compare trap : '1001' > '200' = False ← WRONG (alphabetic)
Correct int compare : 1001  > 200  = True
Fix                 : 1001
```

52 Other CSV shapes

Not every "CSV" uses commas or has a header row. These are optional, but you will meet them:

File looks like	Tell DictReader
`1001;Prudhvi Akella;…` (European exports)	`delimiter=';'`
Tab-separated (`.tsv`)	`delimiter='\t'`
`1001|Prudhvi Akella|…`	`delimiter='|'`
No header row	`fieldnames=['customer_id', 'name', 'city']`
Not sure	`csv.Sniffer().sniff(sample)` guesses the dialect

`io.StringIO(text)` wraps a string so it behaves like an open file — handy for small tests without creating files:

NOTEBOOK CELL 39

```
# ── Semicolon-delimited (common in European exports) ────────
semicolon_csv = """customer_id;name;city
1001;Prudhvi Akella;Rajahmundry
1002;Ravi Kumar;Hyderabad
"""

reader = csv.DictReader(io.StringIO(semicolon_csv), delimiter=';')
for row in reader:
    print(row)
```

OUTPUT

```
{'customer_id': '1001', 'name': 'Prudhvi Akella', 'city': 'Rajahmundry'}
{'customer_id': '1002', 'name': 'Ravi Kumar', 'city': 'Hyderabad'}
```

NOTEBOOK CELL 43

```
# No header row — supply column names with fieldnames=
no_header_csv = """1001,Prudhvi Akella,Rajahmundry
1002,Ravi Kumar,Hyderabad
"""

reader = csv.DictReader(
    io.StringIO(no_header_csv),
    fieldnames=['customer_id', 'name', 'city']   # supply headers manually
)
for row in reader:
    print(row)
```

OUTPUT

```
{'customer_id': '1001', 'name': 'Prudhvi Akella', 'city': 'Rajahmundry'}
{'customer_id': '1002', 'name': 'Ravi Kumar', 'city': 'Hyderabad'}
```

Detecting the delimiter, and commas inside quotes

NOTEBOOK CELL 45

```
# csv.Sniffer — auto-detect delimiter
sample = """customer_id,name,email,city,state
1001,Prudhvi Akella,"prudhvi@example.com,prudhvi1@example2.com",Rajahmundry,AP
1002,Ravi Kumar,ravi@example.com,Hyderabad,TS
1003,Ananya Sharma,ananya@example.com,Bangalore,KA
"""

dialect  = csv.Sniffer().sniff(sample)   # sniff() reads a sample and guesses
print(type(dialect))
print('Detected delimiter:', repr(dialect.delimiter))
print("Quote character:", repr(dialect.quotechar))
```

OUTPUT

```
<class 'type'>
Detected delimiter: ','
Quote character: '"'
```

NOTEBOOK CELL 46

```
# Use the detected dialect in DictReader
reader = csv.DictReader(io.StringIO(sample), dialect=dialect)
for row in reader:
    print(row)
```

OUTPUT

```
{'customer_id': '1001', 'name': 'Prudhvi Akella', 'email':
'prudhvi@example.com,prudhvi1@example2.com', 'city': 'Rajahmundry', 'state': 'AP'}
{'customer_id': '1002', 'name': 'Ravi Kumar', 'email': 'ravi@example.com', 'city': 'Hyderabad',
'state': 'TS'}
{'customer_id': '1003', 'name': 'Ananya Sharma', 'email': 'ananya@example.com', 'city': 'Bangalore',
'state': 'KA'}
```

Prudhvi's email field contains a comma, but it is inside double quotes, so the csv module keeps it as one value. Splitting lines with `line.split(',')` yourself would break this row — use the csv module.

53 Putting it together: a small AI workflow

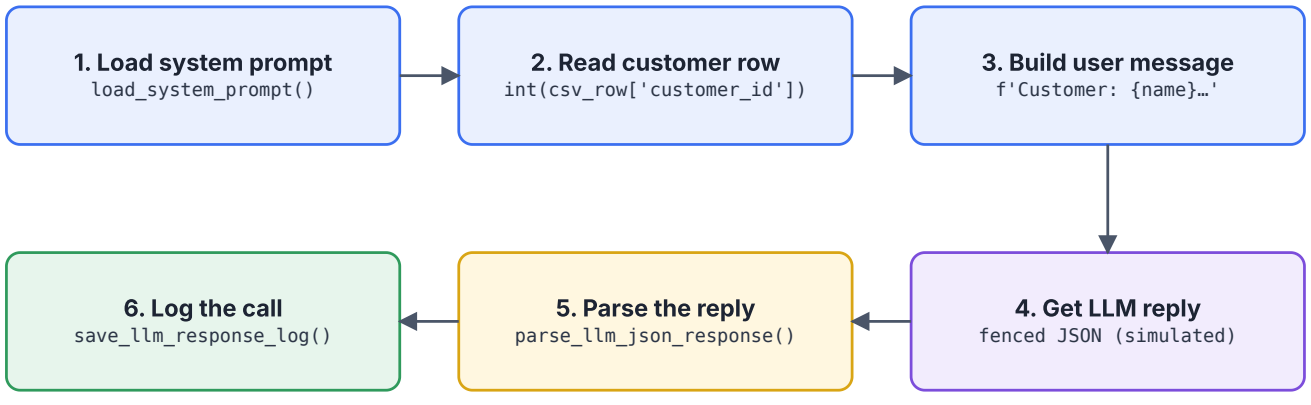


Figure 4.7 The Day 04 mini workflow. Every step reuses something from earlier today.

The notebook ends by chaining today's skills into the shape of a real support bot. The LLM reply is simulated, so it runs without an API key.

NOTEBOOK CELL 50

```
system_prompt = load_system_prompt('sample_prompt.txt')

print(f'Loaded {len(system_prompt)} chars')
print(system_prompt)
```

OUTPUT

```
Loaded 112 chars
## Role
You are a customer support assistant for ShopSmart.

## Task
Answer using ONLY the information provided.
```

NOTEBOOK CELL 52

```
# In production this row comes from csv.DictReader (Section 4)
csv_row       = {'customer_id': '1001', 'name': 'Prudhvi Akella', 'city': 'Rajahmundry'}
customer_id   = int(csv_row['customer_id'])   # CSV always gives str → convert
customer_name = csv_row['name']

print(f'Customer: {customer_name}  id={customer_id}  type={type(customer_id).__name__}')
```

OUTPUT

```
Customer: Prudhvi Akella  id=1001  type=int
```

NOTEBOOK CELL 54

```
user_message = f'Customer: {customer_name}\nQuery: Where is my order #3042?'

print(user_message)
```

OUTPUT

```
Customer: Prudhvi Akella
Query: Where is my order #3042?
```

NOTEBOOK CELL 56

```
# In production this comes back from openai.chat.completions.create()
mock_llm_output = '```json\n{"category": "TRACK_ORDER", "reply": "Your order ORD-3042 is In
Transit.", "confidence": "high"}\n```'

print('Raw LLM output:')
print(mock_llm_output)
```

OUTPUT

````
Raw LLM output:
```json
{"category": "TRACK_ORDER", "reply": "Your order ORD-3042 is In Transit.", "confidence": "high"}
```
````

`.get(key, default)` protects against a reply that leaves a field out:

NOTEBOOK CELL 58

```
parsed     = parse_llm_json_response(mock_llm_output)
category   = parsed.get('category',   'UNKNOWN')
reply      = parsed.get('reply',       'No reply')
confidence = parsed.get('confidence',  'unknown')

print('Parsed dict :', parsed)
print('category    :', category)
print('reply       :', reply)
print('confidence  :', confidence)
```

OUTPUT

```
Parsed dict : {'category': 'TRACK_ORDER', 'reply': 'Your order ORD-3042 is In Transit.',
'confidence': 'high'}
category    : TRACK_ORDER
reply       : Your order ORD-3042 is In Transit.
confidence  : high
```

NOTEBOOK CELL 60

```
save_llm_response_log(user_message, reply)

# Read back to confirm it was written
with open('llm_calls.log', 'r', encoding='utf-8') as f:
    lines = f.readlines()

print(f'llm_calls.log now has {len(lines)} entries')
print('Last entry:', lines[-1].strip())
```

OUTPUT

```
llm_calls.log now has 3 entries
Last entry: {"query": "Customer: Prudhvi Akella\nQuery: Where is my order #3042?", "response": "Your
order ORD-3042 is In Transit."}
```

54 Day 04 recap

Concept	Remember
JSON	Text format for structured data; `true/false/null`, double quotes
`loads` / `dumps`	JSON string ↔ Python (the s = string)
`load` / `dump`	JSON file ↔ Python
`with open(...)`	Closes the file for you; add `encoding='utf-8'`
Modes	`'r'` read · `'w'` overwrite · `'a'` append
JSONL	One JSON object per line — the log format for LLM calls
Fenced replies	Remove the Markdown code fence (backtick lines) before `json.loads()`
`csv.DictReader`	Each row → dict; values are **text**, convert numbers
Other CSVs	`delimiter=`, `fieldnames=`, `csv.Sniffer`, `io.StringIO`

DAY 05

Exceptions, Logging & Retry

Real programs meet bad input, broken JSON and slow networks. Today: catch errors instead of crashing, raise clear errors yourself, record what happened with logging, retry temporary failures, and a first look at tracing.

Notebook: **05_Exception_Handling_Logging_OpenTelemetry.ipynb**

Exceptions | try / except | Specific errors | as error | finally | raise | Logging levels | Bad CSV rows | Retry + backoff | OpenTelemetry

55 What is an exception?

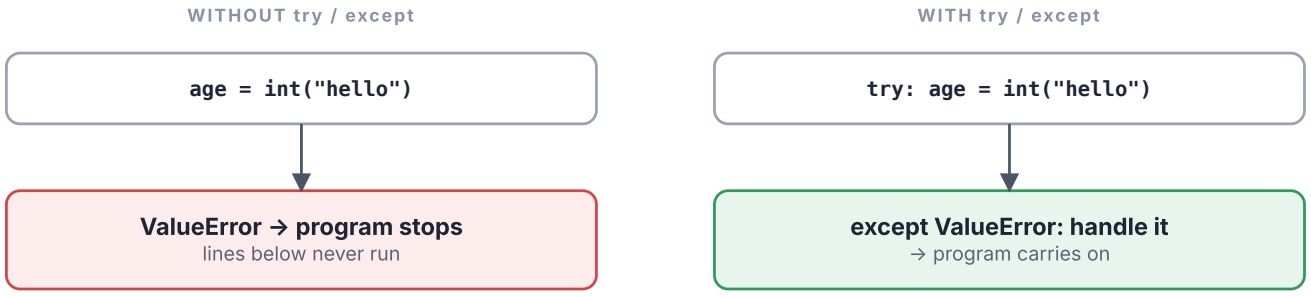


Figure 5.1 Without handling, the first error stops the program. With try / except you decide what happens next.

An **exception** is an error that happens while the program runs — turning `"hello"` into a number, reading a key that isn't there, opening a missing file. Python stops at that line and prints the error **name** and a **message**. Read those two first; they usually tell you exactly what went wrong.

Exception	Example	Meaning
`ValueError`	`int("hello")`	Right type, wrong format
`TypeError`	`len(None)`	Wrong type of value
`KeyError`	`customer["city"]` with no city	Dict key doesn't exist
`FileNotFoundError`	`open("missing.txt")`	File isn't there
`json.JSONDecodeError`	`json.loads("{bad}")`	Text isn't valid JSON

56 try and except

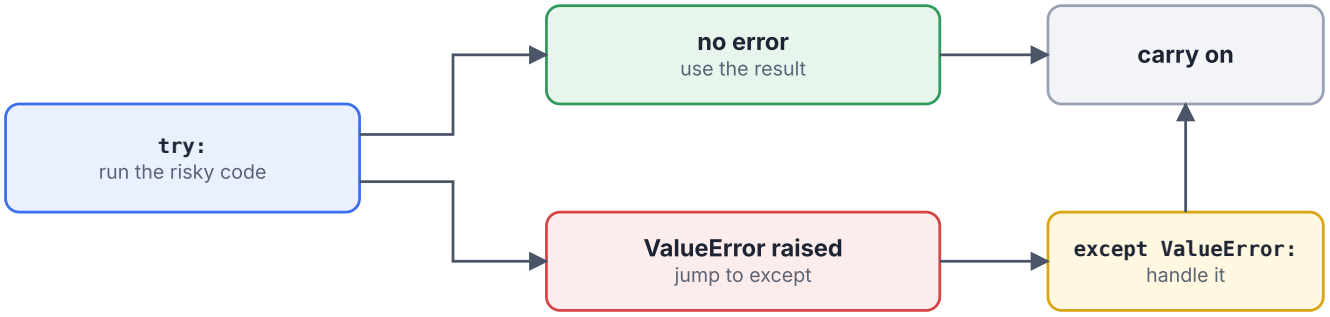


Figure 5.2 Python runs try. If the named error happens, it jumps to except; if not, except is skipped.

Put code that might fail inside `try`. If the error named in `except` happens, Python runs that block instead of stopping. `"3.14"` is not a whole number, so `int()` fails:

NOTEBOOK CELL 8

```
# This conversion fails, so we handle the ValueError.
try:
    number = int("3.14")
    print(number)
except ValueError:
    print("That text is not a valid integer.")
```

OUTPUT

```
That text is not a valid integer.
```

Inside a function: return a safe value

A common pattern: try the work, and on failure print a helpful message and return `None` so the caller can check it.

NOTEBOOK CELL 9

```
# Catch a ValueError when converting text to an integer.
def safe_to_int(value):
    try:
        return int(value)
    except ValueError:
        print(f"Cannot convert {value!r} to an integer.")
        return None

print(safe_to_int("1001"))
print(safe_to_int("hello"))
```

OUTPUT

```
1001
Cannot convert 'hello' to an integer.
None
```

57 Catch the specific errors you expect

A function can have several `except` blocks, one per kind of problem. Here a missing key and a `None` value need different messages (calling `.strip()` on `None` raises `AttributeError`):

NOTEBOOK CELL 10

```
# A dictionary lookup can raise KeyError; a missing or None name needs care.
def get_customer_name(customer, key):
    try:
        value = customer[key]
        return value.strip().title()
    except KeyError:
        print(f"Customer field {key!r} is missing.")
        return "Unknown"
    except AttributeError:
        print(f"Customer field {key!r} has no text value.")
        return "Unknown"

customer = {"name": "prudhvi akella", "email": None}
print(get_customer_name(customer, "name"))
print(get_customer_name(customer, "email"))
print(get_customer_name(customer, "city"))
```

OUTPUT

```
Prudhvi Akella
Customer field 'email' has no text value.
Unknown
Customer field 'city' is missing.
Unknown
```

If both problems get the same treatment, list them in one tuple:

NOTEBOOK CELL 11

```
# Prefer specific exceptions over a broad `except Exception`.
# This function handles only the two errors we expect.
def get_customer_name_safe(customer, key):
    try:
        return customer[key].strip().title()
    except (KeyError, AttributeError):
        return "Unknown"

print(get_customer_name_safe({"name": "ravi kumar"}, "name"))
print(get_customer_name_safe({}, "name"))
```

OUTPUT

```
Ravi Kumar
Unknown
```

NOTEBOOK CELL 12

```
# One except block can handle more than one expected exception type.
def parse_number(value):
    try:
        return float(value)
    except (ValueError, TypeError):
        print(f"Cannot convert {value!r} to a number.")
        return None

print(parse_number("205.21"))
print(parse_number("hello"))
print(parse_number(None))
```

OUTPUT

```
205.21
Cannot convert 'hello' to a number.
None
Cannot convert None to a number.
None
```

Avoid a bare `except:` or `except Exception:` that catches everything. It also hides real bugs — a typo in a variable name would silently return "Unknown" too. Name the errors you expect.

58 Read the error message with `as`

`except SomeError as error` gives you the exception object. Printing it shows Python's message, which says exactly what was wrong and where:

NOTEBOOK CELL 13

```
# TypeError example, handled so the notebook can continue.
try:
    print(float(None))
except TypeError as error:
    print(f"Conversion failed: {error}")
```

OUTPUT

```
Conversion failed: float() argument must be a string or a real number, not 'NoneType'
```

NOTEBOOK CELL 16

```
# json.loads() raises JSONDecodeError when the text is not valid JSON.
bad_json = '{"category": "TRACK_ORDER", missing_quote: "high"}'

try:
    parsed = json.loads(bad_json)
except json.JSONDecodeError as error:
    print(f"Could not read JSON: {error}")
    parsed = {}

print("Parsed value:", parsed)
```

OUTPUT

```
Could not read JSON: Expecting property name enclosed in double quotes: line 1 column 29 (char 28)
Parsed value: {}
```

"line 1 column 29 (char 28)" points to `missing_quote` — in JSON every key needs double quotes. For `JSONDecodeError`, `error.msg` gives the short message without the position.

59 finally: code that always runs

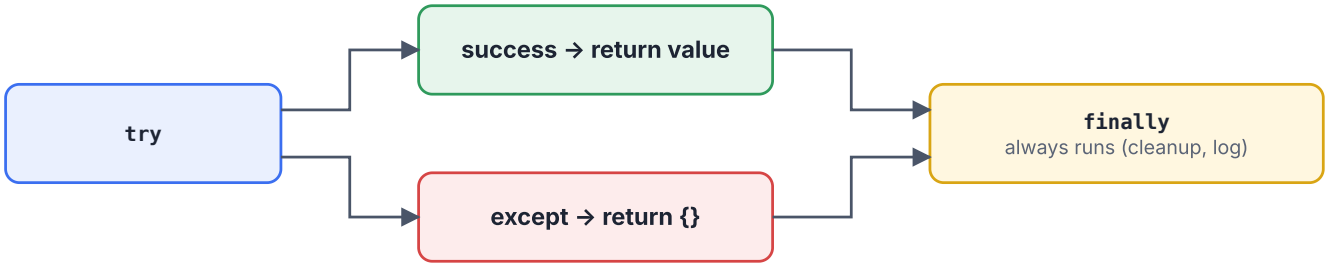


Figure 5.3 Both paths pass through finally — success and failure.

NOTEBOOK CELL 19

```
# A tiny example: finally runs even when the conversion fails.
try:
    number = int("hello")
except ValueError:
    print("Conversion failed.")
finally:
    print("This message always appears.")
```

OUTPUT

```
Conversion failed.
This message always appears.
```

`finally` runs **even when** the `try` or `except` block has already returned. Use it to close things or to record that an attempt finished:

NOTEBOOK CELL 20

```
def parse_json_safely(raw_text):
    try:
        return json.loads(raw_text)
    except json.JSONDecodeError as error:
        print(f"Invalid JSON: {error.msg}")
        return {}
    finally:
        print("JSON parsing attempt finished.")

print(parse_json_safely('{"status": "ok"}'))
print(parse_json_safely('{bad json}'))
```

OUTPUT

```
JSON parsing attempt finished.
{'status': 'ok'}
Invalid JSON: Expecting property name enclosed in double quotes
JSON parsing attempt finished.
{}
```

60 raise: report invalid input yourself

Sometimes your own code spots a problem — a customer ID of `-5` is a valid number but a wrong ID. `raise ValueError("…")` stops the function and hands a clear error to the caller, who decides what to do.

NOTEBOOK CELL 25

```
def validate_customer_id(customer_id):
    try:
        customer_id = int(customer_id)
    except (ValueError, TypeError) as error:
        raise ValueError("Customer ID must be a whole number.") from error

    if customer_id <= 0:
        raise ValueError("Customer ID must be greater than zero.")

    return customer_id

print(validate_customer_id("1001"))

for test_id in ["abc", "-5"]:
    try:
        print(validate_customer_id(test_id))
    except ValueError as error:
        print("Invalid ID:", error)
```

OUTPUT

```
1001
Invalid ID: Customer ID must be a whole number.
Invalid ID: Customer ID must be greater than zero.
```

Input	What happens	Message the caller sees
`"1001"`	Converts, > 0, returned	—
`"abc"`	`int()` fails → we raise our own ValueError	Customer ID must be a whole number.
`"-5"`	Converts, but ≤ 0 → raise	Customer ID must be greater than zero.

`raise … from error` keeps the original error attached, so the full story is still in the traceback when you debug.

How errors travel between functions



Figure 5.4 An error travels up to the caller until some function handles it.

NOTEBOOK CELL 27

```
def parse_customer_id(raw_id):
    customer_id = int(raw_id)
    if customer_id <= 0:
        raise ValueError("Customer ID must be greater than zero.")
    return customer_id

def build_customer_prompt(raw_id):
    try:
        customer_id = parse_customer_id(raw_id)
        return f"Prepare a prompt for customer {customer_id}."
    except ValueError as error:
        return f"Could not build prompt: {error}"

print(build_customer_prompt("1001"))
print(build_customer_prompt("abc"))
```

OUTPUT

```
Prepare a prompt for customer 1001.
Could not build prompt: invalid literal for int() with base 10: 'abc'
```

Rule of thumb: handle an error where you can make a **useful decision**. `parse_customer_id` can't decide what message to show, so it lets the error travel up to `build_customer_prompt`, which can.

61 Logging instead of print



Figure 5.5 The level works like a filter: messages below it are hidden.

`print()` is fine while learning. Real applications use **logging**: every message has a **level**, a timestamp can be added, and you choose how much to see without deleting any lines.

Level	Use it for
`DEBUG`	Detailed information while developing
`INFO`	Normal events — task started, request succeeded
`WARNING`	Something odd happened, the program continues — a default was used
`ERROR`	An operation failed
`CRITICAL`	A serious problem for the whole app

NOTEBOOK CELL 30

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format="%(levelname)s: %(message)s",
    force=True,
)
log = logging.getLogger("day05")

log.debug("Detailed information for debugging")
log.info("Program started")
log.warning("A value was missing; using a default")
log.error("An operation failed")
log.critical("A serious problem occurred")
```

OUTPUT

```
DEBUG: Detailed information for debugging
INFO: Program started
WARNING: A value was missing; using a default
ERROR: An operation failed
CRITICAL: A serious problem occurred
```

`force=True` lets you call `basicConfig` again in the same notebook (normally only the first call counts). `format="%(levelname)s: %(message)s"` decides how each line looks.

Logging inside a function

Pass values with `%s` placeholders: `log.error("Response was not valid JSON: %s", error.msg)`.

NOTEBOOK CELL 31

```
def safe_parse_llm_json(raw_response):
    log.info("Trying to parse the response.")

    try:
        result = json.loads(raw_response)
        log.info("Response parsed successfully.")
        return result
    except json.JSONDecodeError as error:
        log.error("Response was not valid JSON: %s", error.msg)
        return {}
    finally:
        log.info("Parsing attempt finished.")

print(safe_parse_llm_json('{"category": "TRACK_ORDER"}'))
print(safe_parse_llm_json('{bad json}'))
```

OUTPUT

```
INFO: Trying to parse the response.
INFO: Response parsed successfully.
INFO: Parsing attempt finished.
INFO: Trying to parse the response.
ERROR: Response was not valid JSON: Expecting property name enclosed in double quotes
INFO: Parsing attempt finished.
{'category': 'TRACK_ORDER'}
{}
```

Filtering by level

Set the level to `WARNING` for a quieter run — the debug and info lines simply don't appear:

```
logging.basicConfig(
    level=logging.WARNING,
    format="%(levelname)s: %(message)s",
    force=True,
)
log = logging.getLogger("day05")

log.debug("This debug message is hidden.")
log.info("This info message is hidden.")
log.warning("This warning is shown.")
log.error("This error is shown.")
```

```
OUTPUT
WARNING: This warning is shown.
ERROR: This error is shown.
```

62 Use case: one bad CSV row must not stop the job



Figure 5.6 Each row is tried on its own. A bad row becomes a warning, not a crash.

CSV rows (Day 04) arrive as text and are sometimes incomplete. Catch the problems per row, log a warning, and keep processing the rest:

NOTEBOOK CELL 34

```
def parse_customer_csv_row(row):
    try:
        return {
            "customer_id": int(row["customer_id"]),
            "name": f'{row["first_name"]} {row["last_name"]}'.strip(),
            "email": row["email"].lower().strip(),
            "city": row["city"],
        }
    except KeyError as error:
        log.warning("Missing CSV column: %s", error)
        return {}
    except (ValueError, AttributeError) as error:
        log.warning("Invalid CSV value: %s", error)
        return {}

valid_row = {
    "customer_id": "1001", "first_name": "Ravi", "last_name": "Kumar",
    "email": "RAVI@example.com", "city": "Hyderabad"
}
missing_column = {"customer_id": "1002", "first_name": "Ananya"}
bad_id = {
    "customer_id": "not-a-number", "first_name": "Ananya",
    "last_name": "Sharma", "email": "a@example.com", "city": "Pune"
}

print(parse_customer_csv_row(valid_row))
print(parse_customer_csv_row(missing_column))
print(parse_customer_csv_row(bad_id))
```

OUTPUT

```
WARNING: Missing CSV column: 'last_name'
WARNING: Invalid CSV value: invalid literal for int() with base 10: 'not-a-number'
{'customer_id': 1001, 'name': 'Ravi Kumar', 'email': 'ravi@example.com', 'city': 'Hyderabad'}
{}
{}
```

Row	Problem	Caught by
`valid_row`	none — id converted to int, email lower-cased	—
`missing_column`	no `last_name` key	`KeyError` → warning
`bad_id`	`int("not-a-number")` fails	`ValueError` → warning

The warnings appear before the printed dicts because logging writes to a different stream (stderr) than `print` (stdout). Jupyter shows each stream as it arrives.

63 Retry with exponential backoff

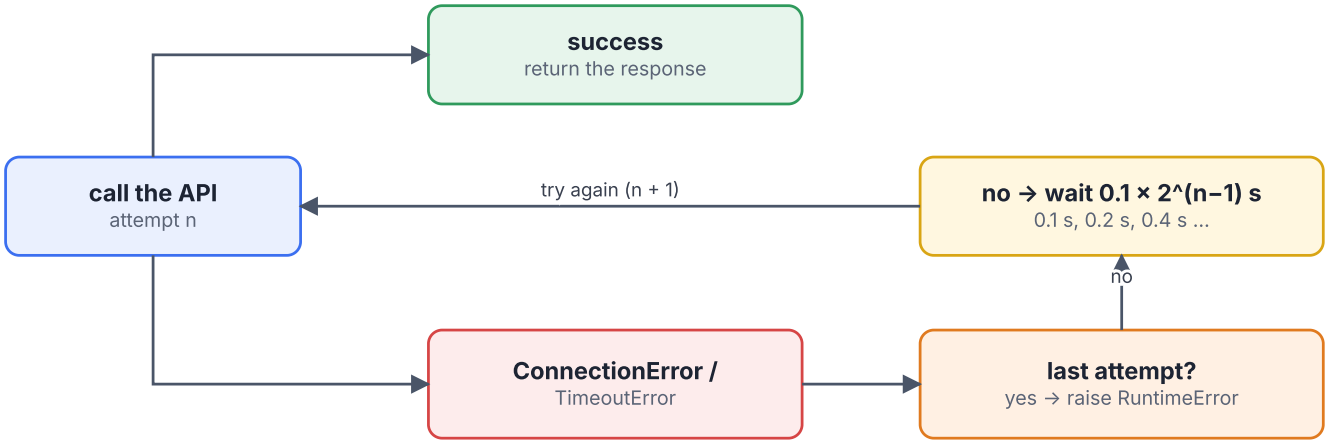


Figure 5.7 Retry with exponential backoff: wait a little longer after every failure, give up after max_attempts.

Some failures are **temporary**: a timeout, a dropped connection, an LLM API saying "too many requests". Trying again after a short wait often works. **Exponential backoff** doubles the wait each time, so a busy server gets room to recover.

The notebook simulates an API that fails twice and then succeeds:

NOTEBOOK CELL 37

```
def simulate_api_call(attempt):
    # Simulate two temporary failures, then a successful response.
    if attempt == 0:
        raise ConnectionError("Temporary connection problem")
    if attempt == 1:
        raise TimeoutError("The request took too long")
    return '{"status": "success"}'
```

The retry loop is a `while` loop (Day 02) with `try/except` inside:

NOTEBOOK CELL 38

```
def call_with_retry(max_attempts=3):
    attempt = 0

    while attempt < max_attempts:
        try:
            log.info("Attempt %s of %s", attempt + 1, max_attempts)
            response = simulate_api_call(attempt)
            log.info("Request succeeded.")
            return response
        except (ConnectionError, TimeoutError) as error:
            attempt += 1
            if attempt == max_attempts:
                log.error("All attempts failed: %s", error)
                raise RuntimeError("Request failed after retries.") from error

            wait_seconds = 0.1 * (2 ** (attempt - 1))
            log.warning("Attempt failed: %s. Retrying in %.1f seconds.", error, wait_seconds)
            time.sleep(wait_seconds)

response = call_with_retry()
print("Final response:", response)
```

OUTPUT

```
INFO: Attempt 1 of 3
WARNING: Attempt failed: Temporary connection problem. Retrying in 0.1 seconds.
INFO: Attempt 2 of 3
WARNING: Attempt failed: The request took too long. Retrying in 0.2 seconds.
INFO: Attempt 3 of 3
INFO: Request succeeded.
Final response: {"status": "success"}
```

Attempt	simulate_api_call	Then
1	`ConnectionError`	wait 0.1×2^0 = **0.1 s**
2	`TimeoutError`	wait 0.1×2^1 = **0.2 s**
3	returns `'{"status": "success"}'`	return it — done

With only one attempt allowed, the first failure is final and the caller catches the `RuntimeError`:

NOTEBOOK CELL 40

```
# The retry demo above succeeds after two temporary failures.
# If every attempt failed, RuntimeError could be caught like this:
try:
    response = call_with_retry(max_attempts=1)
    print("Final response:", response)
except RuntimeError as error:
    print("Request could not be completed:", error)
```

OUTPUT

```
INFO: Attempt 1 of 1
ERROR: All attempts failed: Temporary connection problem
Request could not be completed: Request failed after retries.
```

Retry only errors that can fix themselves (timeouts, connection drops, rate limits). Invalid input like a bad customer ID will fail the same way every time — retrying just wastes time and money.

64 A first look at OpenTelemetry (optional)

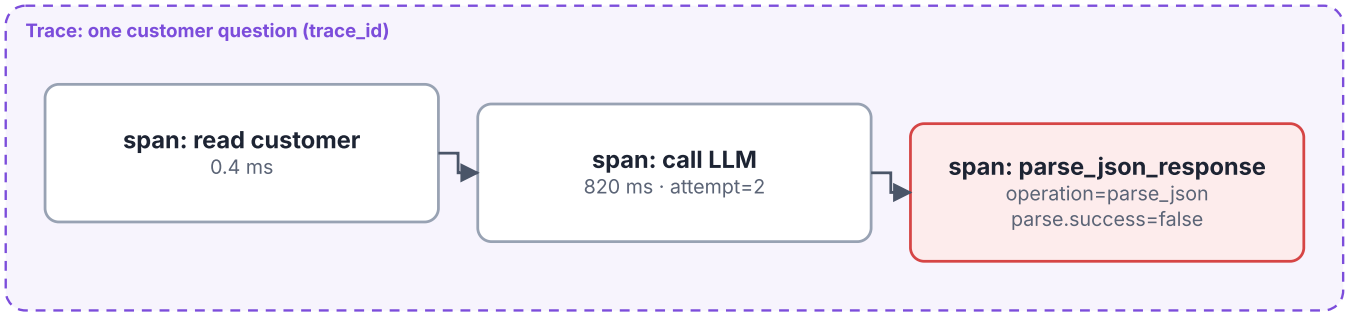


Figure 5.8 A trace is one request's journey. Each step is a span, with attributes and any errors attached (example values).

Logging tells you **what** happened. **Tracing** shows **how a request moved** through your app, step by step, and how long each step took. **OpenTelemetry (OTel)** is the open standard for this.

Word	Meaning
Trace	The journey of one request through the application
Span	One step in that journey — parse a reply, call an API
Attribute	A detail attached to a span — operation name, attempt number

It needs two packages (unlike `logging`, which is built in): `%pip install opentelemetry-api opentelemetry-sdk`. The demo prints each finished span to the notebook:

NOTEBOOK CELL 46

```
# Optional demo: create a trace and one span, then print the span to the output.
# Run the installation step above before running this cell.

from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import SimpleSpanProcessor, ConsoleSpanExporter

# Configure a tracer that prints finished spans to the console.
provider = TracerProvider()
provider.add_span_processor(SimpleSpanProcessor(ConsoleSpanExporter()))
trace.set_tracer_provider(provider)

tracer = trace.get_tracer("day05-demo")

def parse_response_with_trace(raw_response):
    with tracer.start_as_current_span("parse_json_response") as span:
        span.set_attribute("operation", "parse_json")

        try:
            result = json.loads(raw_response)
            span.set_attribute("parse.success", True)
            log.info("JSON response parsed successfully.")
            return result
        except json.JSONDecodeError as error:
```

```
            span.set_attribute("parse.success", False)
            span.record_exception(error)
            span.set_status(trace.Status(trace.StatusCode.ERROR, "Invalid JSON"))
            log.error("Could not parse JSON response.")
            return None

print(parse_response_with_trace('{"status": "ok"}'))
print(parse_response_with_trace('{bad json}'))
```

OUTPUT (SHORTENED)

```
INFO: JSON response parsed successfully.
{
    "name": "parse_json_response",
    "context": { "trace_id": "0x3dff7eae…", "span_id": "0x3ae28973…" },
    "start_time": "2026-11-02T02:05:54.470650Z",
    "end_time":   "2026-11-02T02:05:54.472442Z",
    "status": { "status_code": "UNSET" },
    "attributes": { "operation": "parse_json", "parse.success": true },
    "events": [],
    …
}
{'status': 'ok'}
ERROR: Could not parse JSON response.
{
    "name": "parse_json_response",
    "context": { "trace_id": "0x16b8436a…", "span_id": "0x009b7347…" },
    "status": { "status_code": "ERROR", "description": "Invalid JSON" },
    "attributes": { "operation": "parse_json", "parse.success": false },
    "events": [ { "name": "exception",
                  "attributes": { "exception.type": "json.decoder.JSONDecodeError",
                                  "exception.message": "Expecting property name enclosed in double
quotes: …",
                                  "exception.stacktrace": "Traceback (most recent call last): …" } }
],
    …
}
None
```

Call	What it adds
`start_as_current_span("…")`	Starts a span for this operation; it ends when the `with` block ends
`set_attribute(k, v)`	Searchable context, e.g. `parse.success = false`
`record_exception(error)`	Attaches the exception type, message and stack trace
`set_status(ERROR, …)`	Marks the span as failed

> OTel does not replace logging — the example still logs. In production the spans go to a tracing tool instead of the screen. Learn exceptions and logging first; treat this as a preview.

65 Day 05 recap and practice

Concept	Remember
Exception	An error while running; read its name and message first

Concept	Remember
`try / except`	Run risky code; handle the errors you expect
Specific exceptions	Name them: `except (KeyError, ValueError):` — not a bare `except`
`as error`	Get the message: `print(error)`, `error.msg` for JSON
`finally`	Always runs — cleanup and "attempt finished" logs
`raise`	Signal bad input yourself; `from error` keeps the cause
`logging`	Levels DEBUG < INFO < WARNING < ERROR < CRITICAL; the level is a filter
Bad CSV rows	Catch per row, log a warning, keep going
Retry + backoff	Only for temporary errors; wait 0.1, 0.2, 0.4 s ...; give up after N
OpenTelemetry	Traces and spans show a request's path; works alongside logging

Practice (from the notebook)

1. Convert `"42"` to an integer inside `try/except`.
2. Handle a missing `"city"` key in a dictionary.
3. Write a function that raises `ValueError` when a price is negative.
4. Log a warning when a CSV row is missing a required field.
5. Explain why a timeout may be worth retrying, while invalid input usually isn't.

WRAP-UP

All five days in one script

A short program that uses something from every Day: read customers from CSV, skip a bad row, route the question, build the prompt, call a (fake) LLM that times out once, retry, parse its fenced JSON and log the result.

Notebook: **shopsmart_pipeline.py (new — not in the notebooks)**

Day 01 f-string | Day 02 routing | Day 03 functions | Day 04 CSV + JSON | Day 05 try / retry / log

66 The script

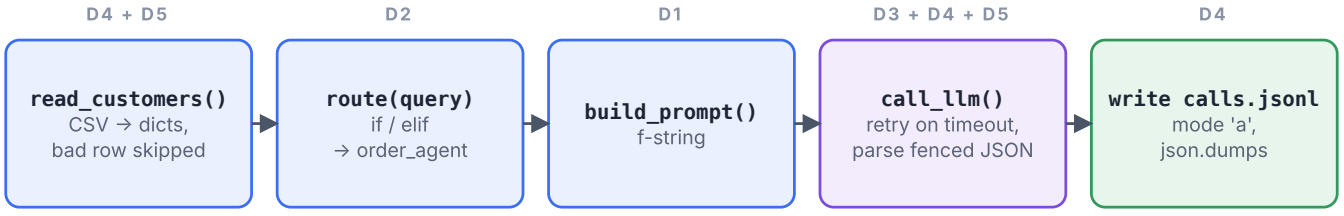


Figure 6.1 What happens for each customer in the script.

Save this as `shopsmart_pipeline.py` and run `python shopsmart_pipeline.py`. Every line uses only what Days 01–05 taught. The comment at the end of each function says which Day it comes from.

```
import csv, io, json, logging, time

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
log = logging.getLogger("shopsmart")

CSV_TEXT = """customer_id,name,city
1001,Prudhvi Akella,Rajahmundry
abc,Bad Row,Nowhere
1002,Ravi Kumar,Hyderabad
"""


def read_customers(text):                                   # Day 04 + Day 05
    customers = []
    for row in csv.DictReader(io.StringIO(text)):
        try:
            customers.append({"id": int(row["customer_id"]), "name": row["name"]})
        except (KeyError, ValueError) as error:
            log.warning("Skipping bad row %s: %s", row["customer_id"], error)
    return customers


def route(query):                                           # Day 02
    q = query.lower()
    if "cancel" in q or "refund" in q:
```

```
        return "human_agent"
    elif "where" in q or "track" in q:
        return "order_agent"
    return "general_agent"


def build_prompt(customer, query):                          # Day 01
    return f"Customer: {customer['name']} (id {customer['id']})\nQuery: {query}"


def fake_llm(prompt, attempt):                              # stands in for a real API
    if attempt == 0:
        raise TimeoutError("LLM took too long")
    return '```json\n{"reply": "Your order is In Transit."}\n```'


def parse_llm_json(raw):                                    # Day 04
    cleaned = raw.strip()
    if cleaned.startswith("```"):
        cleaned = cleaned.split("\n", 1)[-1].rsplit("```", 1)[0]
    return json.loads(cleaned)


def call_llm(prompt, max_attempts=3):                       # Day 03 + Day 05
    for attempt in range(max_attempts):
        try:
            return parse_llm_json(fake_llm(prompt, attempt))
        except TimeoutError as error:
            wait = 0.1 * 2 ** attempt
            log.warning("Attempt %s failed: %s", attempt + 1, error)
            time.sleep(wait)
    raise RuntimeError("LLM failed after retries")


query = "Where is my order #3042?"
for customer in read_customers(CSV_TEXT):
    agent = route(query)
    answer = call_llm(build_prompt(customer, query))
    with open("calls.jsonl", "a", encoding="utf-8") as f:
        f.write(json.dumps({"id": customer["id"], "agent": agent, "reply": answer["reply"]}) +
"\n")
    print(f"{customer['name']:15s} -> {agent} -> {answer['reply']}")
```

OUTPUT (TERMINAL, THEN CALLS.JSONL)

```
WARNING: Skipping bad row abc: invalid literal for int() with base 10: 'abc'
WARNING: Attempt 1 failed: LLM took too long
Prudhvi Akella  -> order_agent -> Your order is In Transit.
WARNING: Attempt 1 failed: LLM took too long
Ravi Kumar      -> order_agent -> Your order is In Transit.
{"id": 1001, "agent": "order_agent", "reply": "Your order is In Transit."}
{"id": 1002, "agent": "order_agent", "reply": "Your order is In Transit."}
```

67 Read it like this

Line(s)	What happens	Day
`csv.DictReader(io.StringIO(text))`	CSV text → one dict per row	04

Line(s)	What happens	Day
`try: int(row["customer_id"])` / `except (KeyError, ValueError)`	Row `abc` can't become a number → warning, skipped	05
`route(query)`	Lower-case, then first matching rule → `order_agent`	02
`build_prompt()`	f-string fills in name and id	01
`fake_llm(prompt, attempt)`	Raises `TimeoutError` on attempt 0, then returns fenced JSON	—
`for attempt in range(max_attempts)`	Retry loop with a default of 3; wait 0.1 s, 0.2 s ...	02 + 03 + 05
`parse_llm_json()`	Remove the code fence, `json.loads()`	04
`open("calls.jsonl", "a")`	Append one JSON line per customer	04
`{customer['name']:15s}`	Pad the name so the arrows line up	01

Try these changes: add a `1003` customer to the CSV; make `fake_llm` fail on attempts 0, 1 and 2 and watch `RuntimeError`; add a `cancel` question and check it routes to `human_agent`; add a `"city"` field to the prompt.

In a real app `fake_llm` becomes a call to an LLM API, and the timeout comes from the network instead of an `if`. The rest of the structure stays exactly the same.